

UNIVERSITÉ CADI AYYAD
FACULTÉ DES SCIENCES ET TECHNIQUES DE MARRAKECH
DÉPARTEMENT D'INFORMATIQUE

MaskanTech Plateforme de Gestion Immobilière

Location · Vente · Rendez-vous · Messagerie

Projet de Fin Module

Réalisateurs du projet :

Salmane ELOUZI

Filière : SIT

Hajar TANANI

Filière : IRISI

Encadrante :

Mme. Sara QASIMI

7 juin 2026

Déclaration

Ce projet intitulé « *MaskanTech — Plateforme de Gestion Immobilière* » a été réalisé dans le cadre du Projet de Fin module de notre cycle d'ingénieur à la Faculté des Sciences et Techniques de Marrakech. Il représente pour nous une approche concrète et approfondie des enjeux liés au développement web full-stack et à la digitalisation du secteur immobilier marocain.

Par la présente, nous, en tant qu'auteurs de ce travail, acceptons pleinement les clauses de confidentialité relatives aux données que nous avons été amenés à manipuler tout au long de cette étude. Nous précisons également que les conclusions et recommandations présentées dans ce rapport n'engagent en aucune manière la responsabilité de notre encadrante, Madame Sara QASIMI.

Nous attestons sur l'honneur que l'ensemble des travaux présentés dans ce document ont été réalisés par nos propres moyens, et que toutes les sources consultées ont été dûment référencées dans la section bibliographique.

Fait à Marrakech, le 7 juin 2026

Remerciements

Nous souhaitons avant tout exprimer notre profonde reconnaissance envers notre encadrante, Madame Sara QASIMI, qui nous a accompagnés tout au long de ce projet de fin d'études avec bienveillance, disponibilité et rigueur scientifique. Son approche pédagogique, qui incarne parfaitement le principe « *Implique-moi et je comprendrai* », a été déterminante dans notre progression et dans la qualité du travail accompli.

Son encadrement exigeant mais toujours constructif, ses retours précis et ses orientations stratégiques nous ont permis de faire le lien entre les fondements théoriques du génie logiciel et leur mise en œuvre pratique dans un contexte de développement d'application réelle. Ce projet est en grande partie l'aboutissement de ses précieux conseils et de son suivi régulier tout au long du semestre.

Nos remerciements s'adressent également à nos camarades de promotion pour les échanges enrichissants et l'entraide tout au long de ce parcours, ainsi qu'à nos familles dont le soutien moral constant a été une source de motivation inépuisable.

Résumé

Ce projet de fin module présente la conception et le développement de , une plateforme web complète dédiée à la gestion immobilière au Maroc. L'application permet la mise en relation entre agents immobiliers, propriétaires et clients dans le cadre d'opérations de location et de vente de biens immobiliers, en centralisant l'ensemble des interactions dans un environnement numérique sécurisé et ergonomique.

offre une interface moderne de type *blanc et doré* développée avec le framework Laravel 13 (PHP 8.5+), TailwindCSS et Alpine.js. La plateforme intègre un système de gestion multi-rôles (Administrateur, Agent immobilier, Client), un module de recherche avancée avec filtres multicritères, un système de prise de rendez-vous, une messagerie interne entre utilisateurs, ainsi qu'une gestion complète de galeries d'images pour chaque bien immobilier. Sur le plan technique, la base de données MySQL comprend 12 tables interconnectées gérant les utilisateurs, les propriétés, les images, les caractéristiques, les favoris, les rendez-vous, les messages et les notifications.

Mots-clés

Laravel	PHP	MySQL	Immobilier	Plateforme web	Gestion multi-rôles
TailwindCSS	Blade	Alpine.js	MVC		

Liste des abréviations

API	Application Programming Interface
BDD	Base de Données
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DTO	Data Transfer Object
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
MVC	Model-View-Controller (équivalent anglais de MCV)
ORM	Object-Relational Mapping
PHP	PHP : Hypertext Preprocessor
REST	Representational State Transfer
SGBDR	Système de Gestion de Base de Données Relationnelle
SIT	Systèmes d'Information et Technologies
SQL	Structured Query Language
UCA	Université Cadi Ayyad
UML	Unified Modeling Language
URL	Uniform Resource Locator

Table des matières

Déclaration	1
Remerciements	2
Résumé	3
Liste des abréviations	4
1 Introduction générale	10
1.1 Contexte général	10
1.2 Motivation du projet	10
1.3 Définition du problème	11
1.4 Organisation du rapport	11
2 Présentation du projet	12
2.1 Étude de l'existant	12
2.1.1 Panorama des plateformes immobilières au Maroc	12
2.1.2 Tableau comparatif	13
2.1.3 Limites des solutions existantes	13
2.2 Solution proposée : MaskanTech	14
2.2.1 Architecture multi-rôles	14
2.2.2 Fonctionnalités principales	15
2.2.3 Design system	15
2.3 Démarche méthodologique	16
2.3.1 Analyse des besoins	16
2.3.2 Conception	16
2.3.3 Développement itératif	16
2.4 Planification	17
3 Analyse et conception	18
3.1 Introduction	18
3.2 Identification des entités	18
3.2.1 Entité User (Utilisateur)	18
3.2.2 Entité Property (Bien immobilier)	18
3.2.3 Entité PropertyImage (Image de bien)	19
3.2.4 Entité PropertyFeature (Caractéristique)	19
3.2.5 Entité Appointment (Rendez-vous)	19
3.2.6 Entité Message	19
3.2.7 Entité BlogPost (Article de blog)	19

3.2.8	Entité <code>BlogComment</code> (Commentaire)	19
3.2.9	Entité <code>Contact</code>	19
3.2.10	Entité <code>Notification</code>	20
3.3	Identification des associations	20
3.4	MCD — Modèle Conceptuel de Données	21
3.5	MLD — Modèle Logique de Données	21
3.6	Description détaillée des tables	22
3.6.1	Table <code>users</code>	22
3.6.2	Table <code>properties</code>	22
3.6.3	Table <code>appointments</code>	23
3.6.4	Table <code>messages</code>	24
3.6.5	Table pivot <code>property_feature_property</code>	25
3.7	Contraintes d'intégrité	25
3.7.1	Contraintes au niveau de la base de données	25
3.7.2	Contraintes au niveau applicatif	25
3.8	Conclusion	26
4	Outils et technologies utilisés	27
4.1	Introduction	27
4.2	Outils côté Backend	27
4.2.1	PHP 8.5	27
4.2.2	Laravel 13	28
4.2.3	MySQL 8.0	30
4.2.4	JSON	31
4.3	Environnement de développement	32
4.3.1	XAMPP / Apache	32
4.3.2	Visual Studio Code	33
4.3.3	Git et GitHub	33
4.4	Outils côté Frontend	36
4.4.1	HTML5 et Blade	36
4.4.2	CSS3 et TailwindCSS	36
4.4.3	Alpine.js	37
4.4.4	Chart.js	38
4.4.5	Vite	38
4.5	Conclusion	38
5	Présentation du site web	40
5.1	Introduction	40
5.2	Interfaces d'authentification	40
5.2.1	Page de connexion	40
5.2.2	Page d'inscription	41
5.2.3	Réinitialisation du mot de passe	42
5.3	Interfaces publiques	43
5.3.1	Page principale — Liste des biens	43
5.3.2	Sidebar de filtres	43
5.3.3	Page de détail d'un bien	44
5.3.4	Page de contact	46
5.4	Interfaces locataire & Etudiant	47
5.4.1	Dashboard Etudiant	47

5.4.2	Page des favoris	47
5.4.3	Page des rendez-vous (locataire)	48
5.4.4	Messagerie — Vue locataire	49
5.4.5	Page de profil	50
5.5	Interfaces propriétaire	51
5.5.1	Dashboard propriétaire	51
5.5.2	Formulaire de publication d’une annonce	51
5.5.3	Gestion des rendez-vous (propriétaire)	53
5.6	Interfaces administrateur	53
5.6.1	Dashboard administrateur	53
5.6.2	Gestion des utilisateurs	54
5.6.3	Gestion de tous les biens	55
5.6.4	Gestion des inscriptions propriétaires	56
5.6.5	Page d’accueil et présentation de l’équipe	57
5.7	Conclusion	59
6	Conclusion générale	61
6.1	Synthèse des travaux	61
6.2	Apports personnels	61
6.3	Limites identifiées	62
6.4	Perspectives d’amélioration	62

Table des figures

3.1	Modèle Conceptuel de Données (MCD) de	21
3.2	Modèle Logique de Données (MLD) de — généré avec MySQL Workbench	21
4.1	Logo PHP	28
4.2	Logo Laravel	29
4.3	Logo MySQL	31
4.4	Logo XAMPP	32
4.5	Logo Visual Studio Code	33
4.6	Logo Git	34
4.7	Graphe Git du projet depuis VS Code	34
4.8	Logo GitHub	35
4.9	Logo TailwindCSS	37
5.1	Page de connexion de — /login	41
5.2	Page d'inscription de — /register	41
5.3	Formulaire de réinitialisation du mot de passe	42
5.4	Formulaire de réinitialisation du mot de passe	42
5.5	Page principale de liste des biens immobiliers — /biens	43
5.6	Sidebar de filtres avancés de	44
5.7	Page de détail d'une annonce immobilière — /biens/{slug}	45
5.8	Section formulaire de demande de visite — page de détail	46
5.9	Page de contact public — /contact	46
5.10	Tableau de bord du locataire — /dashboard	47
5.11	Page des favoris du locataire — /favoris	48
5.12	rendez-vous du locataire — /rendez-vous	48
5.13	Liste des conversations — /messages	49
5.14	Conversation avec un propriétaire — /messages/{user}	50
5.15	Page d'édition du profil utilisateur — /profil	50
5.16	Tableau de bord du propriétaire — /dashboard	51
5.17	Formulaire de publication d'une annonce — /biens/creer	52
5.18	Formulaire de publication d'une annonce — /biens/creer	52
5.19	Formulaire de modification d'une annonce — /biens/{slug}/modifier	53
5.20	Gestion des demandes de visite — vue propriétaire	53
5.21	Tableau de bord de l'administrateur — /admin/	54
5.22	Interface de gestion des utilisateurs — /admin/utilisateurs	55
5.23	Gestion de tous les biens immobiliers — /admin/biens	56
5.24	Validation des propriétaires — /admin/owners/verification	57
5.25	Page d'accueil de	58
5.26	Présentation de l'équipe — Elouzi & Tanani	59

Liste des tableaux

2.1	Comparaison des plateformes immobilières existantes au Maroc	13
2.2	Fonctionnalités principales de MaskanTech	15
2.3	Planning de réalisation du projet MaskanTech (Gantt)	17
3.1	Associations entre les entités de MaskanTech	20
3.2	Structure de la table users	22
3.3	Structure de la table properties	23
3.4	Structure de la table appointments	24
3.5	Structure de la table messages	24
3.6	Structure de la table pivot property_feature_property	25
4.1	Composants Laravel utilisés dans MaskanTech	30

Chapitre 1

Introduction générale

1.1 Contexte général

La transformation numérique constitue aujourd’hui l’un des phénomènes les plus structurants de notre époque. Elle traverse l’ensemble des secteurs d’activité, redessinant profondément les modes de production, de distribution et de consommation des services. Dans ce contexte de mutation accélérée, le secteur immobilier marocain se trouve à la croisée de deux dynamiques majeures : une demande croissante portée par l’urbanisation et la croissance démographique d’une part, et une pression vers la modernisation et la digitalisation des pratiques d’autre part.

Selon les données du Haut-Commissariat au Plan (HCP), le Maroc enregistre chaque année un besoin net en logements estimé à plus de 125 000 unités, avec un déficit cumulé dépassant le million d’unités dans les zones urbaines [1]. Les villes de Casablanca, Marrakech, Rabat, Tanger et Agadir concentrent la grande majorité de cette demande. Parallèlement, le parc immobilier marocain se diversifie avec l’émergence de nouveaux types de biens : résidences de standing, appartements en zones périurbaines, bureaux dans les quartiers d’affaires et terrains à bâtir.

Pourtant, malgré cette effervescence du marché, les pratiques de mise en relation entre offre et demande restent souvent traditionnelles. Les agences immobilières classiques, les annonces dans la presse écrite et le bouche-à-oreille demeurent des canaux majeurs de transaction. Les plateformes numériques existantes, bien qu’en développement, présentent des lacunes importantes en termes de fonctionnalités, d’ergonomie et d’accessibilité pour les acteurs du marché.

1.2 Motivation du projet

C’est dans ce contexte que s’inscrit *maskan*, dont le nom est inspiré du mot arabe *maskan* signifiant « logement » ou « habitat ». Ce projet vise à concevoir et développer une plateforme web complète qui réponde aux besoins réels des trois acteurs principaux du marché immobilier marocain : les agents immobiliers, les propriétaires et les clients.

L’ambition de *maskan* est double. D’une part, offrir aux agents immobiliers un outil professionnel de gestion de leurs annonces, de leurs rendez-vous et de leurs communications avec les clients. D’autre part, proposer aux clients une expérience de recherche moderne, intuitive et complète, leur permettant de trouver le bien correspondant à leurs critères, de contacter directement l’agent concerné et de planifier une visite sans quitter la plateforme.

La plateforme s'appuie sur des technologies modernes et éprouvées : le framework PHP Laravel 13 pour le backend, TailwindCSS et Alpine.js pour le frontend, et MySQL pour la persistance des données. Le choix d'une architecture MVC rigoureuse garantit la maintenabilité et l'évolutivité du système.

1.3 Définition du problème

Avant de présenter la solution retenue, il convient de formaliser la problématique à laquelle répond .

Problématique centrale

Comment concevoir et développer une plateforme web immobilière centralisée, accessible, multi-rôles et dotée d'outils de gestion modernes, permettant de fluidifier les interactions entre agents immobiliers, propriétaires et clients sur le marché marocain, tout en garantissant la sécurité des données et une expérience utilisateur de qualité ?

Cette problématique soulève plusieurs questions secondaires :

- Comment gérer efficacement différents profils utilisateurs avec des droits et des espaces distincts, sans compromettre la sécurité ?
- Comment faciliter la recherche de biens avec des filtres pertinents et une expérience utilisateur intuitive et réactive ?
- Comment intégrer un système de rendez-vous et de messagerie directement dans la plateforme, sans dépendance à des outils tiers ?
- Comment assurer la maintenabilité et l'évolutivité du système face à une croissance potentiellement rapide du nombre d'utilisateurs et d'annonces ?

1.4 Organisation du rapport

Ce rapport est organisé en quatre chapitres complémentaires qui couvrent l'ensemble du cycle de développement de :

- **Chapitre 1 — Introduction générale et Présentation du projet** : contexte détaillé, étude comparative des solutions existantes, fonctionnalités retenues, démarche méthodologique et planification.
- **Chapitre 2 — Analyse et conception** : modélisation de la base de données (MCD et MLD), description des tables et de leurs relations, contraintes d'intégrité.
- **Chapitre 3 — Outils et technologies utilisés** : présentation des frameworks, langages, environnements de développement et outils de gestion de version mobilisés.
- **Chapitre 4 — Présentation de l'application** : interfaces utilisateurs illustrées par des captures d'écran commentées, en distinguant les espaces client, agent et administrateur.

Chapitre 2

Présentation du projet

2.1 Étude de l'existant

2.1.1 Panorama des plateformes immobilières au Maroc

Le marché marocain des plateformes immobilières en ligne compte plusieurs acteurs. Une analyse comparative de ces solutions permet d'identifier les lacunes auxquelles se propose de répondre.

Mubawab.ma

Fondée en 2010, Mubawab est la plateforme immobilière la plus connue au Maroc. Elle propose un catalogue étendu d'annonces couvrant l'ensemble du territoire national. Son interface est moderne et elle dispose d'un outil d'estimation des prix au mètre carré. Cependant, elle ne propose pas de messagerie intégrée entre l'acheteur et l'agent, ni de système de rendez-vous centralisé. La prise de contact se fait exclusivement par téléphone ou email externe, sans traçabilité au sein de la plateforme.

Avito.ma

Avito est une plateforme généraliste de petites annonces qui propose une section immobilière. Son audience est très large grâce à son positionnement multi-catégories. Cependant, sa nature généraliste lui confère peu de spécialisation immobilière : les filtres de recherche sont limités, il n'existe pas de système de rôles différenciés (agent vs particulier vs client), et l'expérience utilisateur est peu adaptée aux besoins spécifiques du secteur.

Sarouty.ma

Sarouty se concentre principalement sur la location et propose une carte interactive permettant de visualiser les biens géographiquement. Cette fonctionnalité est appréciable mais l'interface générale de la plateforme est datée. Elle ne propose pas de tableau de bord pour les agents permettant de suivre leurs performances, ni de système de messagerie intégrée.

Kifach.com

Kifach est une plateforme d'annonces gratuites qui inclut une section immobilière. Son interface est basique et elle ne propose aucun outil de gestion avancé. Elle s'adresse prin-

ciatement aux particuliers souhaitant publier une annonce sans passer par une agence.

2.1.2 Tableau comparatif

Critère	Mubawab	Avito	Sarouty	Kifach	MaskanTech
Multi-rôles	Non	Non	Non	Non	Oui
Messagerie intégrée	Non	Non	Non	Non	Oui
Système de RDV	Non	Non	Non	Non	Oui
Dashboard agent	Partiel	Non	Non	Non	Oui
Favoris client	Oui	Non	Oui	Non	Oui
Galerie photos	Oui	Oui	Oui	Non	Oui
Filtres avancés	Oui	Partiel	Partiel	Non	Oui
Gratuit agents	Non	Oui	Non	Oui	Oui

TABLE 2.1 – Comparaison des plateformes immobilières existantes au Maroc

2.1.3 Limites des solutions existantes

L'analyse comparative révèle plusieurs lacunes communes aux plateformes immobilières marocaines actuelles :

- **Absence de gestion multi-rôles** : aucune des plateformes existantes ne distingue clairement les rôles Administrateur, Agent et Client avec des espaces et des droits différenciés. Cette distinction est pourtant fondamentale pour une gestion professionnelle des annonces et des transactions.
- **Absence de système de rendez-vous intégré** : la prise de contact se fait systématiquement par téléphone ou email externe. Il n'existe aucune traçabilité des demandes de visite, aucun suivi du statut (en attente, confirmé, refusé, terminé) et aucune centralisation du calendrier des visites pour les agents.
- **Messagerie absente ou rudimentaire** : les échanges entre acheteurs ou locataires et agents ne sont pas centralisés dans la plateforme. Cette absence force les utilisateurs à basculer sur des canaux de communication externes, perdant ainsi la traçabilité des échanges.
- **Tableaux de bord insuffisants** : les agents n'ont pas accès à une vue synthétique sur leurs annonces, leurs statistiques de vues, leurs rendez-vous à venir et leurs performances globales.
- **Coûts élevés pour les professionnels** : les solutions professionnelles comme Mubawab sont payantes et leurs tarifs peuvent constituer un frein pour les petites agences immobilières.

2.2 Solution proposée : MaskanTech

se positionne comme une réponse globale à l'ensemble de ces lacunes. La plateforme repose sur trois principes fondateurs :

2.2.1 Architecture multi-rôles

La plateforme distingue trois types d'utilisateurs, chacun disposant d'un espace personnalisé avec des droits et des fonctionnalités adaptés à ses besoins.

Administrateur Bénéficie d'une supervision globale de la plateforme : gestion de tous les utilisateurs (validation des agents, modification des rôles, suppression de comptes), gestion de tous les biens (modération, suppression), accès aux statistiques globales (graphiques d'activité, métriques par mois), gestion des messages de contact reçus et paramétrage de la plateforme.

Agent immobilier Dispose d'un espace professionnel complet : publication et gestion de ses propres annonces immobilières, suivi des demandes de visite reçues (confirmation, refus, calendrier), messagerie avec les clients, et tableau de bord personnel avec statistiques (nombre de biens, vues totales, rendez-vous du mois).

Client Accède à un espace orienté recherche et suivi : recherche avancée avec filtres multi-critères, gestion de sa liste de favoris, demande de rendez-vous de visite, messagerie avec les agents et consultation de l'historique de ses visites et biens récemment consultés.

2.2.2 Fonctionnalités principales

Module	Description	Rôles
Gestion des biens	CRUD complet, galerie jusqu'à 10 photos, caractéristiques (piscine, garage, jardin...), lien vidéo YouTube/Vimeo	Admin, Agent
Recherche avancée	Filtres : type, ville, prix min/max, chambres, statut ; tri : prix, date, surface ; pagination 12 biens/page	Tous
Rendez-vous	Demande, confirmation, refus, statut (pending/confirmed/ refused/completed), calendrier agent	Agent, Client
Messagerie	Chat interne, historique des conversations, indicateur de messages non lus dans la navbar	Tous
Favoris	Sauvegarde et gestion des biens favoris, toggle via AJAX	Client
Dashboard Admin	Statistiques globales, graphique Chart.js (biens/mois), flux d'activité filtrable, alertes	Admin
Dashboard Agent	Mes biens, RDV à venir, vues totales, favoris reçus	Agent
Dashboard Client	Favoris, historique RDV, biens récemment consultés	Client
Blog	Articles immobiliers avec commentaires modérés	Admin
Contact	Formulaire de contact public, gestion admin des messages	Tous

TABLE 2.2 – Fonctionnalités principales de MaskanTech

2.2.3 Design system

adopte un design **clair et élégant**, associant des fonds blancs à des accents dorés. Ce choix visuel vise à offrir une expérience chaleureuse, professionnelle et conviviale, tout en attirant l'attention sur les éléments clés de l'interface (annonces, boutons d'action, messages).

La palette de couleurs repose sur des fonds blancs (`#ffffff`) et gris très clair (`#f8f9fa`), un texte gris foncé pour une lisibilité optimale (`#333333`), et une couleur d'accentuation **dorée** (`#d4af37`) pour les éléments interactifs (boutons, liens, icônes). La typographie utilisée alterne entre une police **sans-serif** moderne (comme Inter ou Poppins) pour les titres et une police classique pour le corps du texte, afin de renforcer la convivialité et l'attention de l'utilisateur.

2.3 Démarche méthodologique

2.3.1 Analyse des besoins

La première phase du projet a consisté en une analyse approfondie des besoins fonctionnels et non fonctionnels de la plateforme. Cette analyse a mobilisé plusieurs techniques complémentaires : étude des plateformes concurrentes (présentée en section 2.1), identification des acteurs du système et de leurs interactions, et définition des cas d'utilisation prioritaires.

Les besoins fonctionnels ont été catégorisés selon leur priorité (Must Have, Should Have, Could Have) selon la méthode MoSCoW, permettant de concentrer les efforts de développement sur les fonctionnalités à plus haute valeur ajoutée.

Les besoins non fonctionnels identifiés incluent notamment :

- **Sécurité** : authentification robuste (Laravel Breeze avec vérification email), gestion des rôles et permissions, protection contre les attaques CSRF, validation des entrées côté serveur.
- **Performance** : pagination des résultats, chargement différé des images, mise en cache des routes et des vues.
- **Maintenabilité** : architecture MVC stricte, séparation des responsabilités, code documenté, versionnement avec Git.
- **Responsivité** : interface adaptée aux écrans mobiles, tablettes et desktop grâce à TailwindCSS.

2.3.2 Conception

La phase de conception a porté sur la modélisation de la base de données (présentée en détail au Chapitre 2) et la définition de l'architecture technique générale. Nous avons opté pour une architecture monolithique MVC, adaptée à la taille et à la nature du projet, plutôt qu'une architecture microservices qui aurait introduit une complexité opérationnelle disproportionnée.

2.3.3 Développement itératif

Le développement a suivi une approche itérative organisée en six sprints successifs, chacun produisant un incrément fonctionnel testable :

Sprint 1 Mise en place de l'environnement de développement (XAMPP, VS Code, Git), création du projet Laravel, configuration de la base de données, définition et exécution des migrations, installation de Laravel Breeze pour l'authentification.

Sprint 2 Développement des modèles Eloquent avec leurs relations (hasMany, belongsToMany, etc.), création des factories et seeders pour les données de démonstration, mise en place du middleware de rôles.

Sprint 3 Développement des contrôleurs CRUD (PropertyController, AppointmentController, etc.), définition des routes (web.php), implémentation des FormRequests de validation et des Politiques d'autorisation.

Sprint 4 Développement des vues Blade : layout principal (layouts/app.blade.php), composants réutilisables (x-metric-card, x-status-badge, etc.), mise en place du design system dark monospace.

Sprint 5 Implémentation des fonctionnalités avancées : dashboards différenciés par rôle, système de rendez-vous avec gestion des statuts, messagerie interne, gestion des favoris, galerie photos avec Alpine.js.

Sprint 6 Tests fonctionnels, correction des bugs identifiés, optimisations (réduction des requêtes N+1, mise en cache), déploiement local complet et rédaction du rapport.

2.4 Planification

La planification du projet a été réalisée sous la forme d'un diagramme de Gantt couvrant la durée totale des travaux, soit douze semaines de développement.

Tâche	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
Analyse des besoins												
Conception BDD												
Migrations & Modèles												
Contrôleurs & Routes												
Interfaces Blade												
Fonctionnalités avancées												
Tests & Débogage												
Rédaction du rapport												

TABLE 2.3 – Planning de réalisation du projet MaskanTech (Gantt)

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre présente la modélisation de la base de données de à travers deux représentations complémentaires : le Modèle Conceptuel de Données (MCD) et le Modèle Logique de Données (MLD). Ces deux modèles constituent le fondement sur lequel repose l'ensemble de l'implémentation technique. Une conception rigoureuse de la base de données est en effet déterminante pour la cohérence, les performances et l'évolutivité d'une application web de cette envergure.

3.2 Identification des entités

L'analyse des besoins fonctionnels de a conduit à identifier les entités suivantes, qui correspondent aux objets métier manipulés par la plateforme.

3.2.1 Entité User (Utilisateur)

L'entité centrale du système. Elle représente tout utilisateur inscrit sur la plateforme, qu'il soit administrateur, agent immobilier ou client. La distinction entre ces trois profils est assurée par l'attribut `role`, qui est un type énuméré pouvant prendre les valeurs `admin`, `agent` ou `client`.

Attributs principaux : identifiant unique, nom complet, adresse email (unique), mot de passe haché, rôle, numéro de téléphone, adresse postale, avatar (chemin vers le fichier image), date de vérification de l'email, et timestamps de création et mise à jour.

3.2.2 Entité Property (Bien immobilier)

L'entité principale du domaine métier. Elle représente une annonce immobilière publiée par un agent. Chaque bien est associé à un unique agent (relation plusieurs-à-un avec `User`).

Attributs principaux : identifiant unique, référence à l'agent propriétaire, titre de l'annonce, slug (identifiant URL généré automatiquement à partir du titre), description longue, prix, surface en m², type de bien (maison, appartement, terrain, bureau), nombre de pièces, chambres et salles de bain, adresse complète, ville, code postal, année de construction, statut (disponible, vendu, loué, en construction), indicateur *bien à la une*, compteur de vues, lien vidéo optionnel.

3.2.3 Entité PropertyImage (Image de bien)

Représente une image associée à un bien immobilier. Chaque bien peut disposer de plusieurs images (relation un-à-plusieurs avec **Property**). L'une des images est marquée comme image principale (**is_primary**).

Attributs : identifiant, référence au bien, chemin du fichier image sur le serveur de stockage, indicateur image principale, position dans l'ordre d'affichage de la galerie.

3.2.4 Entité PropertyFeature (Caractéristique)

Représente une caractéristique ou équipement pouvant être associé à un bien : piscine, garage, jardin, climatisation, chauffage central, meublé, ascenseur, interphone, terrasse, cave. La relation entre **Property** et **PropertyFeature** est de type plusieurs-à-plusieurs, modélisée par une table pivot.

3.2.5 Entité Appointment (Rendez-vous)

Représente une demande de visite formulée par un client pour un bien donné. Elle implique trois acteurs : le bien concerné, le client demandeur et l'agent responsable du bien. Le statut évolue selon le cycle de vie de la demande : **pending** (en attente), **confirmed** (confirmé par l'agent), **refused** (refusé) ou **completed** (visite effectuée).

Attributs : identifiant, références au bien, au client et à l'agent, date souhaitée, heure souhaitée, message optionnel du client, statut courant.

3.2.6 Entité Message

Représente un message échangé entre deux utilisateurs de la plateforme (client vers agent ou agent vers client). Chaque message est associé à un expéditeur et un destinataire. Un lien optionnel vers un bien peut contextualiser la conversation. L'indicateur **is_read** permet de gérer les notifications de messages non lus dans la navbar.

Attributs : identifiant, référence à l'expéditeur, référence au destinataire, référence optionnelle au bien concerné, contenu du message, indicateur de lecture.

3.2.7 Entité BlogPost (Article de blog)

Représente un article publié sur la section blog de la plateforme. Ces articles, rédigés par l'administrateur, traitent de sujets liés à l'immobilier (conseils d'achat, tendances du marché, etc.). Un compteur de vues permet de mesurer la popularité de chaque article.

3.2.8 Entité BlogComment (Commentaire)

Représente un commentaire posté par un utilisateur authentifié sur un article de blog. Les commentaires sont soumis à modération : leur statut peut être **pending** (en attente de validation) ou **approved** (publié).

3.2.9 Entité Contact

Représente un message envoyé via le formulaire de contact public de la plateforme. Ces messages sont visibles dans l'interface d'administration. L'indicateur **is_read** permet

à l'administrateur de savoir quels messages ont déjà été consultés.

3.2.10 Entité Notification

Représente une notification interne générée par le système à destination d'un utilisateur : confirmation de rendez-vous, nouveau message reçu, etc. Les données de chaque notification sont stockées au format JSON pour permettre une grande flexibilité dans le contenu transmis.

3.3 Identification des associations

Les entités précédemment identifiées sont liées par les associations suivantes, décrites avec leurs cardinalités.

Entité 1	Cardinalité	Entité 2	Nature
User (agent)	1 – N	Property	Un agent publie plusieurs biens
Property	1 – N	PropertyImage	Un bien a plusieurs images
Property	N – N	PropertyFeature	Un bien a plusieurs caractéristiques
User (client)	N – N	Property (favoris)	Un client sauvegarde plusieurs biens
Appointment	N – 1	Property	Un RDV concerne un bien
Appointment	N – 1	User (client)	Un RDV est demandé par un client
Appointment	N – 1	User (agent)	Un RDV est géré par un agent
Message	N – 1	User (sender)	Un message a un expéditeur
Message	N – 1	User (receiver)	Un message a un destinataire
BlogPost	1 – N	BlogComment	Un article a plusieurs commentaires
Notification	N – 1	User	Une notification cible un utilisateur

TABLE 3.1 – Associations entre les entités de MaskanTech

3.4 MCD — Modèle Conceptuel de Données

Le Modèle Conceptuel de Données (MCD) représente de façon abstraite et indépendante de toute technologie les entités du système et leurs associations. Il constitue la première étape de la modélisation et sert de référence commune entre les membres de l'équipe.

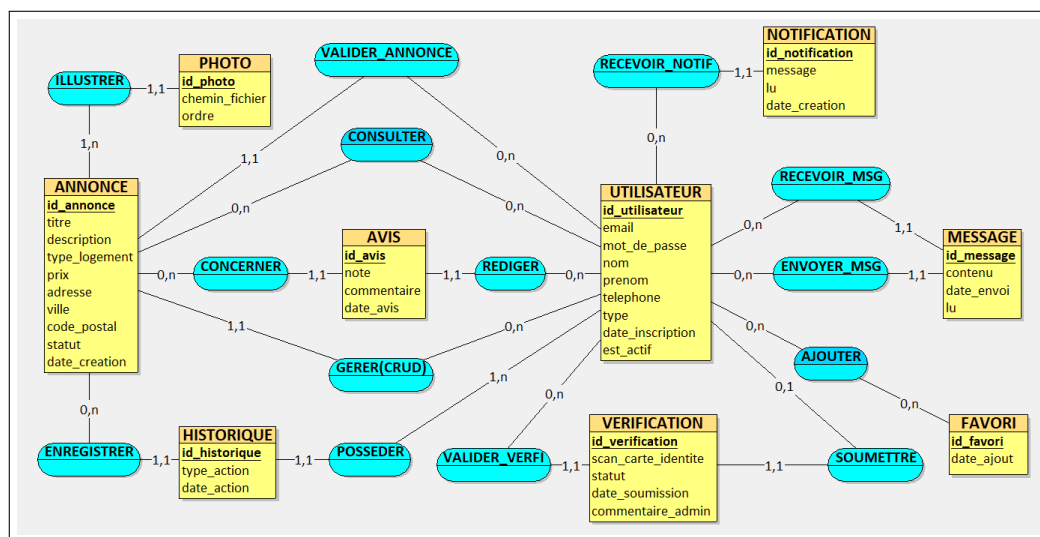


FIGURE 3.1 – Modèle Conceptuel de Données (MCD) de

3.5 MLD — Modèle Logique de Données

Le Modèle Logique de Données (MLD) traduit le MCD en tables relationnelles MySQL. Les clés primaires sont soulignées, les clés étrangères sont précédées du symbole #, et les types énumérés sont indiqués en italique.

Légende : clé primaire #clé étrangère type énuméré

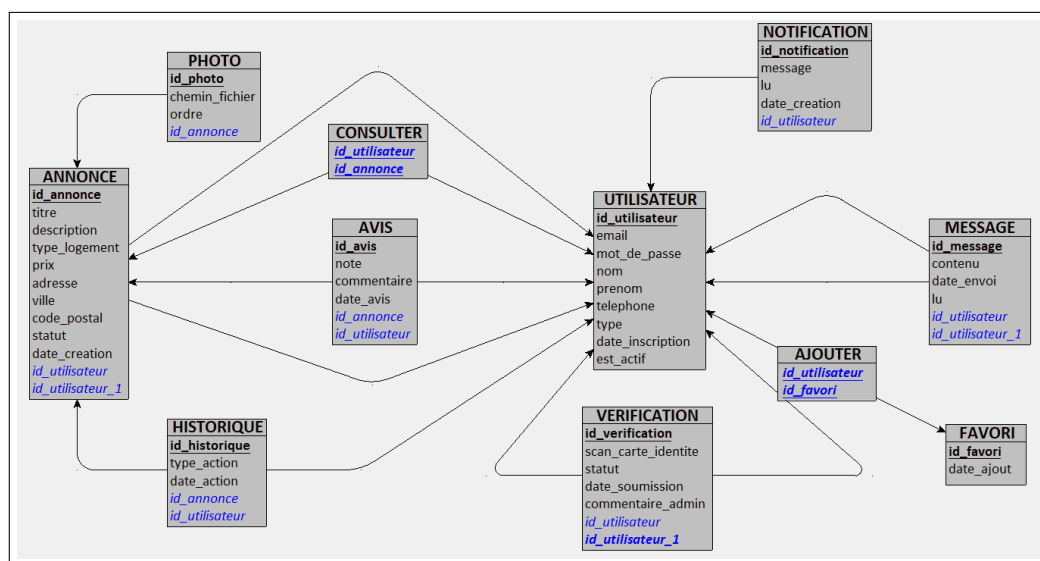


FIGURE 3.2 – Modèle Logique de Données (MLD) de — généré avec MySQL Workbench

3.6 Description détaillée des tables

3.6.1 Table users

La table **users** est la table centrale du système. Elle stocke tous les comptes utilisateurs, quel que soit leur rôle. La colonne **role** est de type **ENUM** avec les valeurs possibles **admin**, **agent** et **client**. La valeur par défaut est **client**, ce qui garantit qu'un utilisateur nouvellement inscrit n'obtient pas de droits étendus sans une action explicite de l'administrateur.

La colonne **email_verified_at** est nullable et contient la date de vérification de l'adresse email. Tant qu'elle est nulle, l'accès aux fonctionnalités protégées est bloqué par le middleware **verified** de Laravel Breeze.

Colonne	Type	Null	Description
id	BIGINT UNSIGNED	Non	Clé primaire auto-incrémentée
name	VARCHAR(255)	Non	Nom complet de l'utilisateur
email	VARCHAR(255)	Non	Email unique, sert d'identifiant
password	VARCHAR(255)	Non	Mot de passe haché (bcrypt)
role	ENUM	Non	admin / agent / client (défaut : client)
phone	VARCHAR(20)	Oui	Numéro de téléphone
address	TEXT	Oui	Adresse postale
avatar	VARCHAR(255)	Oui	Chemin vers l'image de profil
email_verified_at	TIMESTAMP	Oui	Date de vérification email
remember_token	VARCHAR(100)	Oui	Token "se souvenir de moi"
created_at	TIMESTAMP	Oui	Date de création
updated_at	TIMESTAMP	Oui	Date de dernière modification

TABLE 3.2 – Structure de la table **users**

3.6.2 Table properties

La table **properties** est la table principale du domaine métier. La colonne **slug** est générée automatiquement à partir du titre par le modèle Eloquent lors de la création (méthode **booted()**). Elle est unique et utilisée comme clé dans les URLs (**/biens/{slug}**) plutôt que l'identifiant numérique, ce qui améliore le référencement (SEO) et la lisibilité des URLs.

La colonne **views_count** est incrémentée via une requête AJAX à chaque consultation de la page de détail d'un bien, permettant de mesurer la popularité des annonces sans recharger la page.

Colonne	Type	Null	Description
id	BIGINT UNSIGNED	Non	Clé primaire
user_id	BIGINT UNSIGNED	Non	FK vers users (agent)
title	VARCHAR(255)	Non	Titre de l'annonce
slug	VARCHAR(255)	Non	Identifiant URL unique
description	LONGTEXT	Non	Description complète
price	DECIMAL(12,2)	Non	Prix en dirhams
area	DECIMAL(8,2)	Oui	Surface en m ²
type	ENUM	Non	house / apartment / land / office
rooms	TINYINT UNSIGNED	Oui	Nombre de pièces
bedrooms	TINYINT UNSIGNED	Oui	Nombre de chambres
bathrooms	TINYINT UNSIGNED	Oui	Nombre de salles de bain
address	VARCHAR(500)	Non	Adresse complète
city	VARCHAR(100)	Non	Ville (indexée)
postal_code	VARCHAR(10)	Oui	Code postal
year_built	YEAR	Oui	Année de construction
status	ENUM	Non	available / sold / rented / under_construction
is_featured	BOOLEAN	Non	Bien à la une (défaut : false)
views_count	INT UNSIGNED	Non	Compteur de vues (défaut : 0)
video_url	VARCHAR(500)	Oui	Lien YouTube ou Vimeo

TABLE 3.3 – Structure de la table **properties**

3.6.3 Table appointments

La table **appointments** modélise les rendez-vous de visite. Une contrainte importante est que la date du rendez-vous doit être dans le futur au moment de la création, ce qui est validé côté serveur par la règle `after:today` dans `AppointmentRequest`.

Le cycle de vie du statut suit une progression unidirectionnelle : un rendez-vous commence nécessairement à l'état **pending**, puis peut évoluer vers **confirmed** ou **refused** (par l'agent), et finalement vers **completed** une fois la visite effectuée.

Colonne	Type	Null	Description
id	BIGINT UNSIGNED	Non	Clé primaire
property_id	BIGINT UNSIGNED	Non	FK vers properties
client_id	BIGINT UNSIGNED	Non	FK vers users (client)
agent_id	BIGINT UNSIGNED	Non	FK vers users (agent)
date	DATE	Non	Date souhaitée pour la visite
time	TIME	Non	Heure souhaitée
message	TEXT	Oui	Message optionnel du client
status	ENUM	Non	pending / confirmed / refused / completed

TABLE 3.4 – Structure de la table **appointments**

3.6.4 Table messages

La table **messages** implémente la messagerie interne de la plateforme. Chaque enregistrement représente un message unique entre deux utilisateurs. La colonne **property_id** est nullable : elle permet de contextualiser un message en le liant à un bien immobilier spécifique (par exemple, lorsqu'un client envoie un message depuis la page de détail d'un bien).

L'indicateur **is_read** (booléen, défaut **false**) est mis à **true** lorsque le destinataire consulte la conversation. Son agrégat par destinataire permet d'afficher le badge de messages non lus dans la navbar.

Colonne	Type	Null	Description
id	BIGINT UNSIGNED	Non	Clé primaire
sender_id	BIGINT UNSIGNED	Non	FK vers users (expéditeur)
receiver_id	BIGINT UNSIGNED	Non	FK vers users (destinataire)
property_id	BIGINT UNSIGNED	Oui	FK vers properties (contexte optionnel)
message	TEXT	Non	Contenu du message
is_read	BOOLEAN	Non	Lu / non lu (défaut : false)

TABLE 3.5 – Structure de la table **messages**

3.6.5 Table pivot `property_feature_property`

Cette table pivot modélise la relation plusieurs-à-plusieurs entre les biens immobiliers et leurs caractéristiques. Elle ne contient que deux colonnes, toutes deux clés étrangères, formant ensemble une clé primaire composite. Cette structure garantit l'unicité de chaque association bien-caractéristique.

Les caractéristiques disponibles sont pré-chargées en base via le seeder : Piscine, Garage, Jardin, Climatisation, Chauffage central, Meublé, Ascenseur, Interphone, Terrasse, Cave.

Colonne	Type	Description
<code>property_id</code>	BIGINT UNSIGNED	FK vers <code>properties</code>
<code>property_feature_id</code>	BIGINT UNSIGNED	FK vers <code>property_features</code>

TABLE 3.6 – Structure de la table pivot `property_feature_property`

3.7 Contraintes d'intégrité

La cohérence des données est assurée par un ensemble de contraintes d'intégrité définies au niveau de la base de données et renforcées au niveau applicatif.

3.7.1 Contraintes au niveau de la base de données

- **Unicité de l'email** : la colonne `email` de la table `users` dispose d'un index UNIQUE, empêchant deux utilisateurs de s'inscrire avec la même adresse.
- **Unicité du slug** : la colonne `slug` de la table `properties` dispose d'un index UNIQUE. Le modèle Eloquent gère automatiquement les conflits en ajoutant un suffixe numérique.
- **Intégrité référentielle** : toutes les clés étrangères sont déclarées avec des contraintes FOREIGN KEY. La suppression en cascade (CASCADE) est définie pour les relations parent-enfant (ex : supprimer un bien supprime automatiquement ses images). La mise à null (SET NULL) est utilisée pour les relations optionnelles (ex : supprimer un bien met à null le champ `property_id` dans les messages).
- **Unicité des favoris** : la table `favorites` dispose d'un index UNIQUE composite sur (`user_id`, `property_id`), empêchant un utilisateur d'ajouter le même bien deux fois en favoris.
- **Index de performance** : des index simples sont définis sur les colonnes fréquemment utilisées dans les clauses WHERE des requêtes de recherche : `status`, `type` et `city` de la table `properties`; `is_read` de la table `messages`.

3.7.2 Contraintes au niveau applicatif

- **Validation des formulaires** : chaque formulaire de création ou modification est associé à une classe `FormRequest` Laravel (`PropertyRequest`, `AppointmentRequest`) définissant les règles de validation côté serveur.

- **Autorisation** : le middleware `RoleMiddleware` vérifie le rôle de l'utilisateur authentifié avant chaque action. La classe `PropertyPolicy` vérifie qu'un agent ne peut modifier ou supprimer que ses propres biens.
- **Date de rendez-vous future** : la règle `after:today` dans `AppointmentRequest` empêche de créer un rendez-vous dans le passé.
- **Taille des images** : la règle `max:5120` (5 Mo) dans `PropertyRequest` limite la taille de chaque image uploadée.

3.8 Conclusion

Ce chapitre a présenté la modélisation complète de la base de données de , depuis l'identification des entités et de leurs associations jusqu'à la description détaillée des tables et des contraintes d'intégrité. Le modèle obtenu, composé de 12 tables principales et de 2 tables système Laravel, offre une structure cohérente, normalisée et performante pour supporter l'ensemble des fonctionnalités de la plateforme. Le chapitre suivant présentera les outils et technologies utilisés pour l'implémentation concrète de ce modèle.

Chapitre 4

Outils et technologies utilisés

4.1 Introduction

Ce chapitre présente en détail l'ensemble des technologies, frameworks, bibliothèques et outils de développement mobilisés pour la conception et le développement de . Pour chaque outil, nous précisons sa version, son rôle dans le projet et les raisons qui ont motivé son choix par rapport aux alternatives disponibles.

Le choix de la stack technologique a été guidé par trois critères principaux : la maturité et la stabilité de l'outil, la richesse de sa documentation et de sa communauté, et sa cohérence avec les compétences acquises au cours de la formation SIT.

4.2 Outils côté Backend

4.2.1 PHP 8.5

PHP (Hypertext Preprocessor) est un langage de script côté serveur spécialement conçu pour le développement web. Créé par Rasmus Lerdorf en 1994, il est aujourd'hui l'un des langages les plus utilisés au monde pour le développement d'applications web, alimentant des plateformes comme WordPress, Facebook (dans ses premières versions) et Wikipedia.

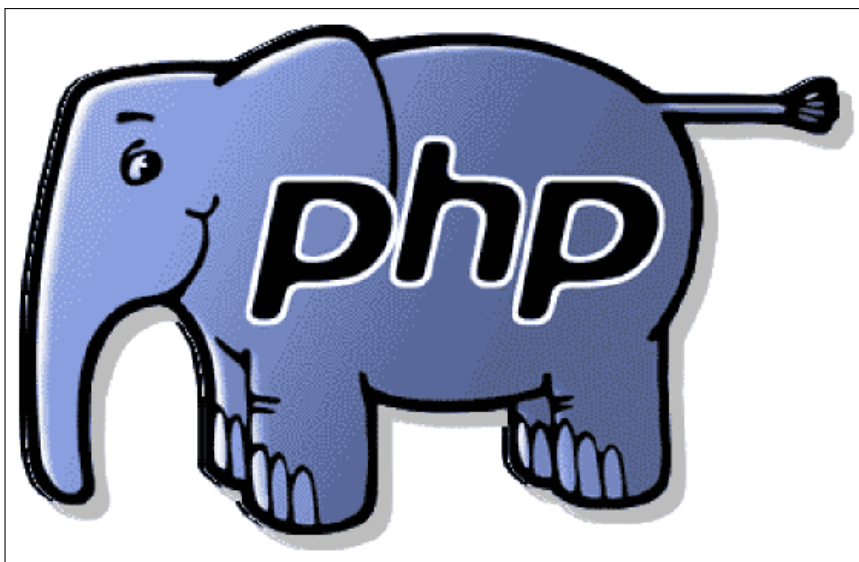


FIGURE 4.1 – Logo PHP

La version 8.5 utilisée dans ce projet apporte des améliorations significatives en termes de performance et d'expressivité du langage. Parmi les fonctionnalités modernes de PHP 8.x exploitées dans :

Match expression Utilisée dans le `DashboardController` pour dispatcher vers la bonne vue selon le rôle de l'utilisateur, de manière plus concise et stricte qu'un `switch`.

Exemple — Match expression

```
return match (Auth::user()->role) {  
    'admin' => $this->admin(),  
    'agent' => $this->agent(),  
    default => $this->client(),  
};
```

Nullsafe operator (`?->`) Permet d'accéder en chaîne à des propriétés ou méthodes d'objets potentiellement nuls, sans vérification explicite. Utilisé notamment dans les vues Blade pour accéder aux données de l'utilisateur authentifié : `auth()->user()->unread_mess`

Arrow functions Closures concises utilisées dans les scopes Eloquent du modèle `Property` pour les requêtes de filtrage.

Named arguments Amélioration de la lisibilité des appels de fonctions avec plusieurs paramètres optionnels.

4.2.2 Laravel 13

Laravel est un framework PHP open source suivant le patron architectural MVC (Modèle-Vue-Contrôleur). Créé par Taylor Otwell en 2011 et aujourd'hui maintenu par une large communauté, il est devenu le framework PHP le plus populaire au monde grâce à son expressivité, sa richesse fonctionnelle et la qualité de sa documentation.

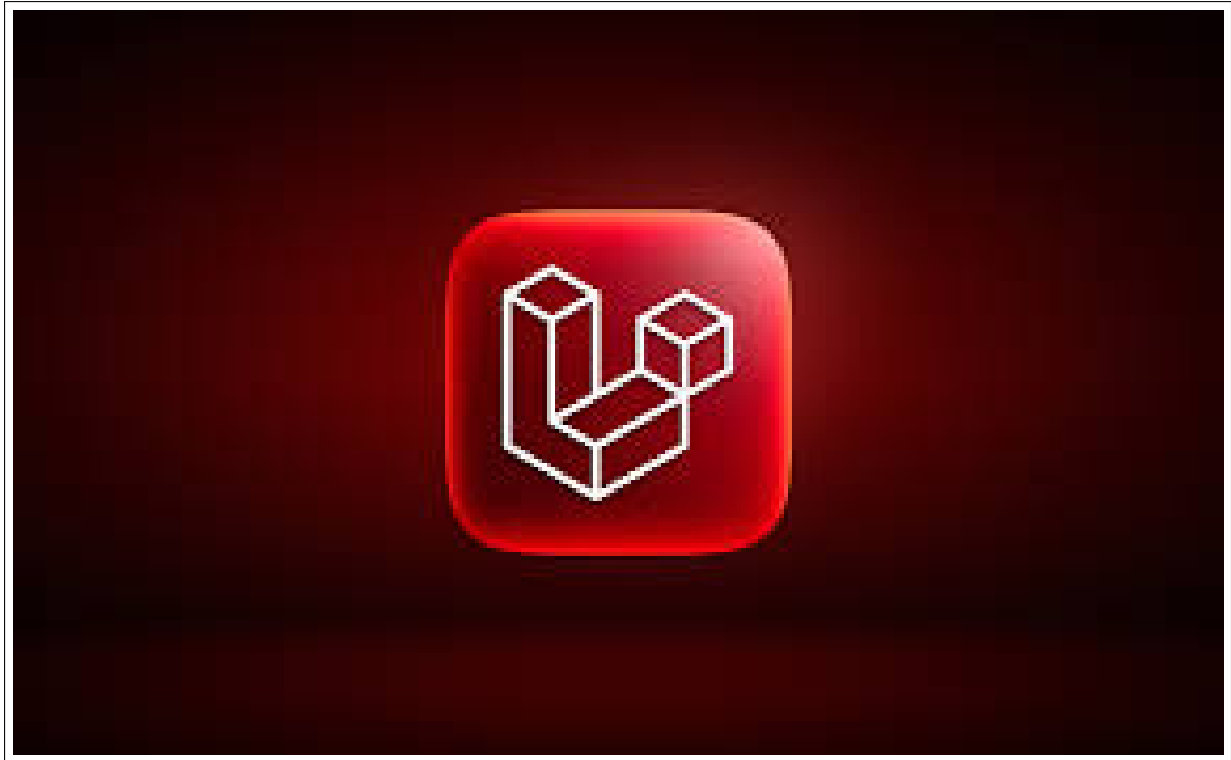


FIGURE 4.2 – Logo Laravel

Le choix de Laravel pour s'est imposé naturellement pour plusieurs raisons :

- **Architecture MVC claire** : la séparation nette entre modèles, vues et contrôleurs facilite la maintenance et l'évolution du code.
- **Eloquent ORM** : la couche d'abstraction de base de données permet de manipuler les données sous forme d'objets PHP, avec une syntaxe expressive et des relations facilement définissables.
- **Système de migrations** : permet de versionner le schéma de la base de données et de le reproduire sur n'importe quel environnement avec une simple commande.
- **Écosystème riche** : Laravel Breeze pour l'authentification, Vite pour la compilation des assets, Artisan pour la génération de code.
- **Sécurité intégrée** : protection CSRF, hachage des mots de passe, protection contre les injections SQL via l'ORM.

Les composants Laravel utilisés dans sont présentés dans le tableau suivant.

Composant Laravel	Utilisation dans MaskanTech
Eloquent ORM	Modèles avec relations complexes (hasMany, belongsToMany, hasManyThrough). Scopes de filtrage dans le modèle Property.
Migrations	3 fichiers de migration créant les 12 tables de la base de données dans un ordre respectant les contraintes de clés étrangères.
Blade	Moteur de templates : layouts, sections, composants anonymes (x-metric-card, x-status-badge, x-feed-item, x-sidebar-label).
Artisan	Génération de code (make :controller, make :model, make :migration), gestion des migrations, vidage du cache, démarrage du serveur.
Laravel Breeze	Authentification complète : inscription, connexion, déconnexion, vérification email, réinitialisation du mot de passe.
FormRequest	PropertyRequest et AppointmentRequest : validation côté serveur avec messages d'erreur personnalisés en français.
Middleware	RoleMiddleware : vérification du rôle de l'utilisateur avant l'accès aux routes protégées. Usage : ->middleware('role :admin').
Policies	PropertyPolicy : contrôle fin de l'autorisation de modification et suppression des biens (seul l'auteur ou l'admin peut agir).
Storage	Gestion des images uploadées (photos de biens, avatars) stockées dans storage/app/public avec lien symbolique vers public/.
Factories / Seeders	UserFactory et PropertyFactory pour les données de démonstration. DatabaseSeeder crée 1 admin, 3 agents, 10 clients, 15 biens.
Vite	Compilation et bundling des assets JavaScript et CSS en production avec hot module replacement en développement.

TABLE 4.1 – Composants Laravel utilisés dans MaskanTech

4.2.3 MySQL 8.0

MySQL est un système de gestion de base de données relationnelle (SGBDR) open source, développé et maintenu par Oracle Corporation. Il s'agit de l'un des SGBDR les plus déployés au monde, particulièrement dans les applications web où il forme avec PHP

le socle de la stack LAMP (Linux, Apache, MySQL, PHP).



FIGURE 4.3 – Logo MySQL

Dans , MySQL stocke l'ensemble des données persistantes de la plateforme. La configuration de la connexion est définie dans le fichier `.env` du projet et exploitée par Laravel via le driver `pdo_mysql`.

Les optimisations de performance appliquées à la base de données incluent :

- **Index sur les colonnes de filtrage** : `status`, `type` et `city` de la table `properties` sont indexés pour accélérer les requêtes de recherche.
- **Index sur les colonnes de tri** : `created_at` et `price` sont indexés pour les opérations de tri fréquentes.
- **Eager loading** : utilisation systématique de `with(['primaryImage', 'favoritedBy'])` dans le contrôleur pour éviter le problème des requêtes N+1.
- **Pagination** : toutes les listes de résultats sont paginées (12 éléments par page) pour limiter la quantité de données chargées en mémoire.

Configuration de la connexion MySQL (`.env`)

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=maskan_tech_db
DB_USERNAME=root
DB_PASSWORD=
```

4.2.4 JSON

JSON (JavaScript Object Notation) est un format léger d'échange de données, facile à lire pour les humains et facile à analyser pour les machines. Dans , il est utilisé à plusieurs niveaux :

- **Colonne data de la table notifications** : stockage des données spécifiques à chaque type de notification (identifiant du bien concerné, nom de l'expéditeur, etc.) sous forme de tableau JSON, sérialisé et désérialisé automatiquement par le casting Eloquent.

- **Réponses API REST** : l'endpoint `/api/properties/search` retourne les résultats de recherche au format JSON pour les appels AJAX.
- **Transmission de données à Chart.js** : les tableaux de données du graphique du dashboard admin sont transmis via la directive `@json($chartData)` de Blade.

4.3 Environnement de développement

4.3.1 XAMPP / Apache

XAMPP est un environnement de développement local multiplateforme comprenant Apache (serveur web), MySQL (base de données), PHP et Perl. Il constitue une solution rapide à installer pour disposer d'un serveur web fonctionnel sur une machine de développement Windows.



FIGURE 4.4 – Logo XAMPP

Dans le cadre de ce projet, XAMPP a été utilisé principalement pour sa composante MySQL, accessible via phpMyAdmin pour la gestion visuelle de la base de données. Le serveur web pour Laravel est assuré par la commande `php artisan serve`, qui démarre un serveur de développement intégré sur le port 8000.

Démarrage du serveur de développement Laravel

```
cd C:\MY_LAB_PHP\Apache24\htdocs\MaskanTech
php artisan serve
# Serveur accessible sur http://127.0.0.1:8000
```

4.3.2 Visual Studio Code

Visual Studio Code (VS Code) est un éditeur de code source développé par Microsoft. Léger, extensible et performant, il est devenu l'environnement de développement intégré (IDE) de facto pour les projets web. Son système d'extensions permet d'adapter l'éditeur à pratiquement n'importe quel langage ou framework.

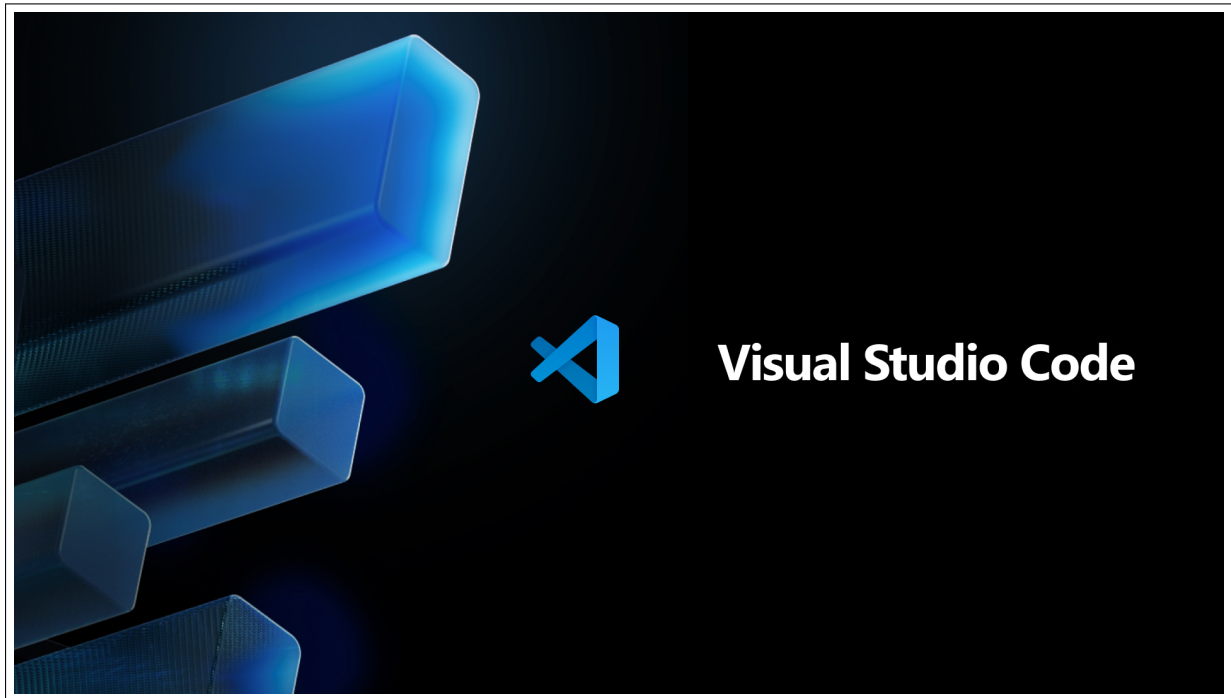


FIGURE 4.5 – Logo Visual Studio Code

Les extensions installées pour le développement de :

PHP Intelephense Fournit l'autocomplétion, la navigation dans le code, la détection d'erreurs de syntaxe et l'analyse statique pour PHP. Indispensable pour travailler efficacement avec Laravel.

Laravel Extension Pack Ensemble d'extensions dédiées à Laravel : snippets Blade, IntelliSense pour les routes et les modèles Eloquent, navigation vers les vues depuis les contrôleurs.

Tailwind CSS IntelliSense Autocomplétion des classes utilitaires TailwindCSS directement dans les fichiers Blade et HTML. Affiche également un aperçu de la couleur pour les classes de couleur.

GitLens Enrichit l'intégration Git de VS Code avec l'affichage de l'historique des commits par ligne (*blame*), la visualisation des diffs et la navigation dans l'historique du projet.

Thunder Client Client HTTP intégré à VS Code permettant de tester les endpoints de l'API REST de directement depuis l'éditeur, sans quitter l'environnement de développement.

4.3.3 Git et GitHub

Git est un système de contrôle de version distribué créé par Linus Torvalds en 2005. Il permet de suivre l'évolution du code source dans le temps, de créer des branches pour tra-

vailler sur des fonctionnalités isolées, et de fusionner les changements de manière contrôlée.



FIGURE 4.6 – Logo Git

Visualisation du graphe Git

L'extension **Git Graph** de VS Code permet de visualiser l'historique des branches et des fusions. La capture ci-dessous illustre la stratégie de branches appliquée dans ce projet :

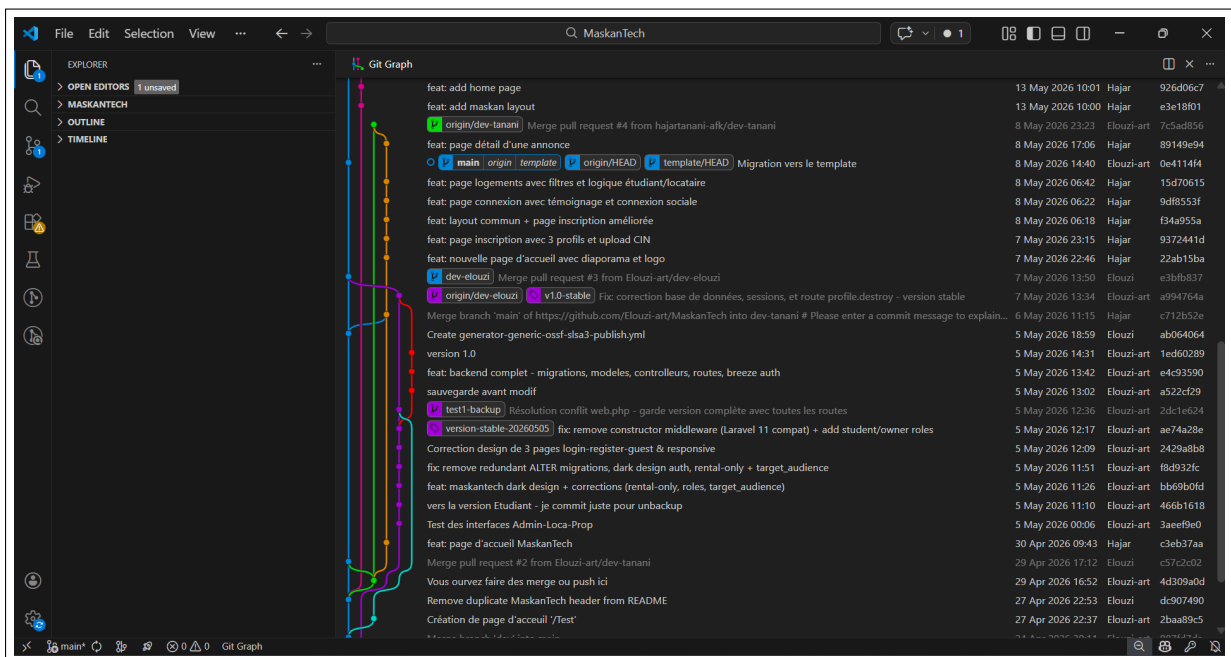


FIGURE 4.7 – Graphe Git du projet depuis VS Code

On peut y observer :

- La branche `main` (stable) représentée en noir/vert.
- La branche `dev-elouzi` (développement principal) avec ses commits de fonctionnalités.
- La branche `dev-tanani` créée à partir de `dev-elouzi` pour isoler le développement de certaines fonctionnalités.
- Les points de fusion (*merge commits*) entre les branches.

GitHub est la plateforme d'hébergement de dépôts Git la plus utilisée au monde. Elle offre en complément des fonctionnalités de collaboration (pull requests, issues, code review) et d'intégration continue.



FIGURE 4.8 – Logo GitHub

Dépôt du projet

Le code source complet du projet est disponible à l'adresse suivante :

```
https://github.com/Elouzi-art/MaskanTech.git
```

Stratégie de branches

La stratégie de branches adoptée dans ce projet :

main Branche principale, contenant le code stable et testé.

dev-elouzi Branche de développement principal où les nouvelles fonctionnalités sont intégrées et testées avant d'être fusionnées dans `main`.

dev-tanani Branche de développement utilisée par Tanani pour ses fonctionnalités, fusionnée régulièrement dans `dev-elouzi`.

Convention de commits

Les commits suivent une convention de messages descriptifs avec un préfixe indiquant la nature du changement : **feat:** (nouvelle fonctionnalité), **fix:** (correction de bug), **refactor:** (refactoring sans changement de comportement). Des tags Git marquent les versions stables : **version-stable-20260505**.

Contenu du dépôt

Le dépôt contient l'intégralité du code source de l'application :

- Code Laravel (Controllers, Models, Views, Migrations, Seeders)
- Configuration Tailwind CSS et Alpine.js
- Scripts de base de données
- Documentation d'installation dans le fichier `README.md`

4.4 Outils côté Frontend

4.4.1 HTML5 et Blade

HTML5 est le langage de balisage standard pour la création de pages web. Dans , le HTML est généré dynamiquement via le moteur de templates Blade de Laravel. Blade offre une syntaxe concise et expressive pour les structures de contrôle (`@if`, `@foreach`, `@auth`), l'héritage de templates (`@extends`, `@section`, `@yield`), les composants réutilisables (`<x-metric-card>`) et l'injection de directives PHP (`@json`, `@csrf`, `@stack`).

4.4.2 CSS3 et TailwindCSS

TailwindCSS est un framework CSS utilitaire qui permet de construire des interfaces directement dans le HTML en appliquant des classes prédéfinies, sans écrire de CSS personnalisé. Contrairement à Bootstrap qui propose des composants pré-stylisés, TailwindCSS propose des utilitaires de bas niveau (espacement, typographie, couleurs, flexbox, grid) que le développeur assemble librement.



FIGURE 4.9 – Logo TailwindCSS

Dans , TailwindCSS est chargé via CDN avec une configuration personnalisée définissant les couleurs et la typographie du design system :

Configuration TailwindCSS personnalisée

```
tailwind.config = {  
  theme: {  
    extend: {  
      colors: {  
'dark-bg': '#0d0d0d',  
'dark-card': '#111111',  
'dark-card2': '#141414',  
'dark-border': '#222222',  
'dark-text': '#d0d0d0',  
'dark-muted': '#666666',  
'dark-dim': '#444444',  
      },  
    fontFamily: {  
      mono: ['"Courier New"', 'monospace'],  
    }  
  }  
}
```

4.4.3 Alpine.js

Alpine.js est un framework JavaScript léger (environ 15 Ko minifié et compressé) qui apporte la réactivité directement dans le HTML via des attributs `x-data`, `x-show`, `x-if`,

etc., sans nécessiter de configuration ou de build step. Il est souvent décrit comme le « Tailwind du JavaScript » en raison de sa philosophie utilitaire.

Dans , Alpine.js est utilisé pour :

- **Menu déroulant utilisateur** dans la navbar : ouverture et fermeture du dropdown au clic (`x-data="{ open: false }", @click.away="open = false"`).
- **Filtres du feed d'activité** sur le dashboard admin : masquage et affichage des éléments selon le type sélectionné.
- **Galerie photos interactive** sur la page de détail d'un bien : navigation entre les images et agrandissement.
- **Messages flash auto-masquables** : disparition automatique après 4 secondes (`x-init="setTimeout(() => show = false, 4000)"`).
- **Modales de confirmation** : affichage conditionnel des boîtes de dialogue avant les actions destructives (suppression d'un bien).

4.4.4 Chart.js

Chart.js est une bibliothèque JavaScript open source permettant de créer des graphiques interactifs et responsifs dans le navigateur via l'élément HTML5 `<canvas>`. Elle supporte de nombreux types de graphiques : barres, lignes, camembert, radar, etc.

Dans , Chart.js est utilisé pour afficher le graphique en barres du dashboard administrateur, représentant le nombre de biens publiés par mois sur les douze derniers mois. Le thème du graphique est personnalisé pour s'intégrer au design dark de la plateforme : fond sombre, bordures indigo, tooltips avec police Courier New.

4.4.5 Vite

Vite est un outil de build frontend moderne développé par Evan You (créateur de Vue.js). Il offre un démarrage quasi-instantané en développement grâce au rechargement à chaud (Hot Module Replacement) et génère des bundles optimisés pour la production.

Laravel intègre nativement Vite depuis la version 9 via le plugin `@vitejs/plugin-laravel`. Dans , Vite compile le fichier `resources/js/app.js` (qui importe Axios pour les requêtes AJAX) et le fichier `resources/css/app.css`.

Commandes Vite utilisées dans le projet

```
npm install    # Installation des dépendances
npm run dev    # Serveur de développement avec HMR
npm run build  # Build de production (génère public/build/)
```

4.5 Conclusion

Ce chapitre a présenté l'ensemble de la stack technologique de , organisée en trois couches complémentaires : le backend (PHP 8.5, Laravel 13, MySQL 8.0), l'environnement de développement (XAMPP, VS Code, Git, GitHub) et le frontend (HTML5/Blade, TailwindCSS, Alpine.js, Chart.js, Vite).

Le choix de Laravel comme framework central offre un équilibre optimal entre productivité de développement, richesse fonctionnelle et maintenabilité du code. La combinaison

TailwindCSS + Alpine.js côté frontend permet de construire des interfaces réactives et esthétiquement cohérentes sans la complexité d'un framework JavaScript lourd. Le chapitre suivant présentera l'application finale à travers ses différentes interfaces.

Chapitre 5

Présentation du site web

5.1 Introduction

Ce chapitre présente les différentes interfaces de à travers des captures d'écran commentées. La plateforme est organisée en trois espaces distincts correspondant aux trois rôles : l'espace public (accessible sans authentification), l'espace client, l'espace agent immobilier et l'espace administrateur. Chaque interface a été conçue en cohérence avec le design system *dark terminal* présenté au chapitre précédent.

L'ensemble des interfaces est accessible après démarrage du serveur local via la commande `php artisan serve`, sur l'adresse `http://127.0.0.1:8000`.

5.2 Interfaces d'authentification

5.2.1 Page de connexion

La page de connexion (`/login`) est le point d'entrée de la plateforme pour les utilisateurs enregistrés. Elle est générée par Laravel Breeze et personnalisée pour s'intégrer au design dark de .

Le formulaire comprend deux champs : l'adresse email et le mot de passe. Une case à cocher « Se souvenir de moi » permet de maintenir la session active. Un lien « Mot de passe oublié » redirige vers le formulaire de réinitialisation.

La protection contre les attaques par force brute est assurée par le système de rate limiting intégré à Laravel Breeze : après cinq tentatives échouées, l'accès est bloqué temporairement et un message d'erreur indique le délai d'attente.

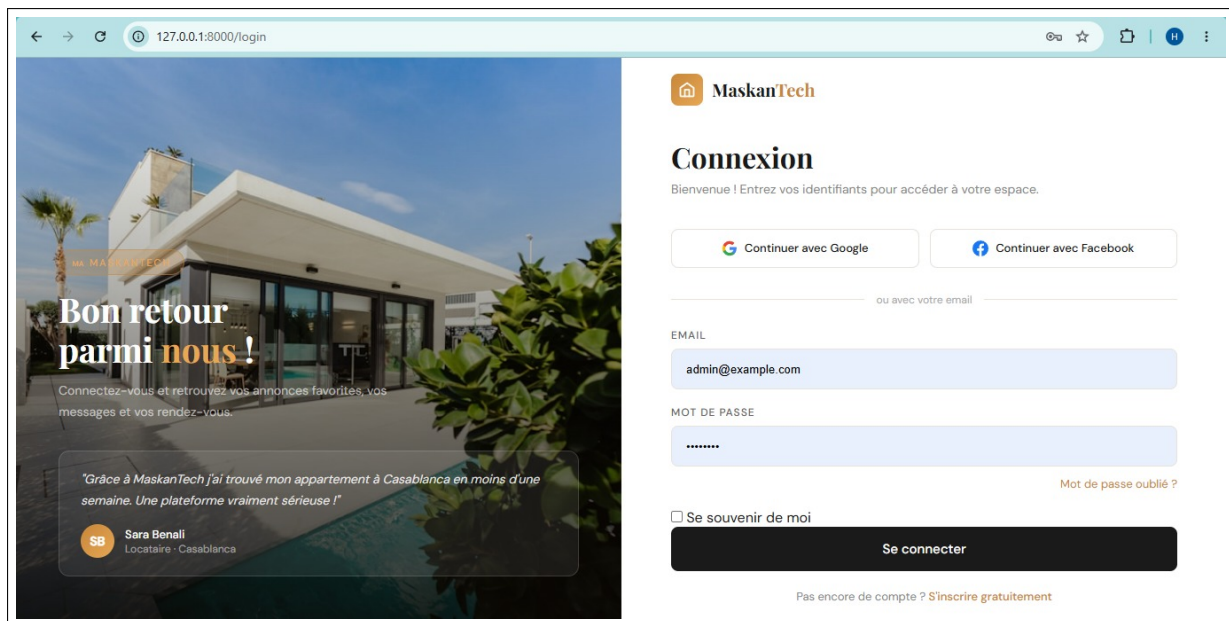


FIGURE 5.1 – Page de connexion de — /login

5.2.2 Page d'inscription

La page d'inscription (/register) permet à un nouvel utilisateur de créer un compte sur la plateforme. Le formulaire collecte le nom complet, l'adresse email, le mot de passe et sa confirmation.

Par défaut, tout nouvel inscrit reçoit le rôle `client`. Pour devenir agent immobilier, l'utilisateur doit contacter l'administrateur qui procédera à la modification de son rôle via l'interface d'administration.

Après inscription, un email de vérification est envoyé à l'adresse fournie. L'accès aux fonctionnalités protégées (dashboard, favoris, rendez-vous, etc.) est conditionné à la vérification de l'email, assurée par le middleware `verified` de Laravel.

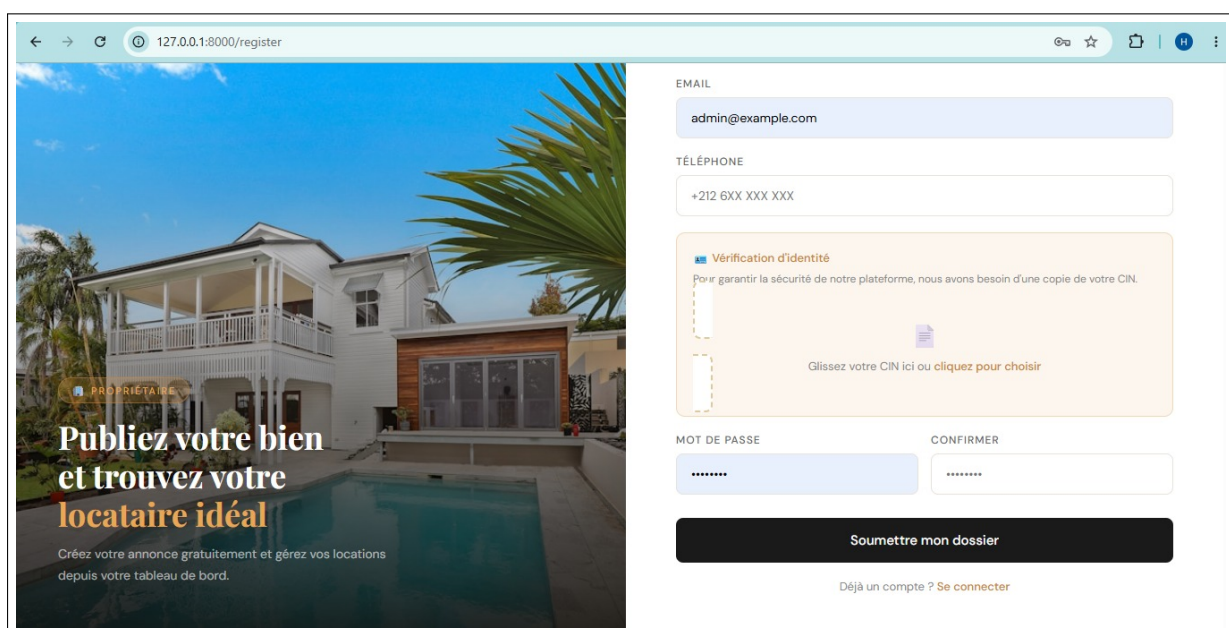
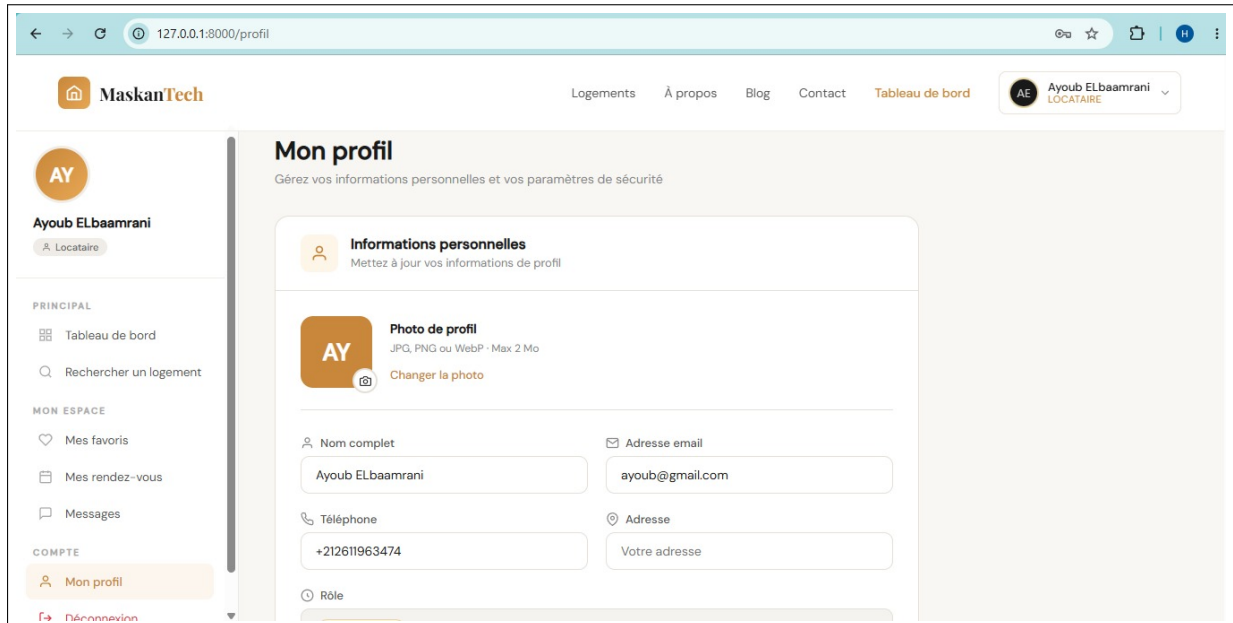


FIGURE 5.2 – Page d'inscription de — /register

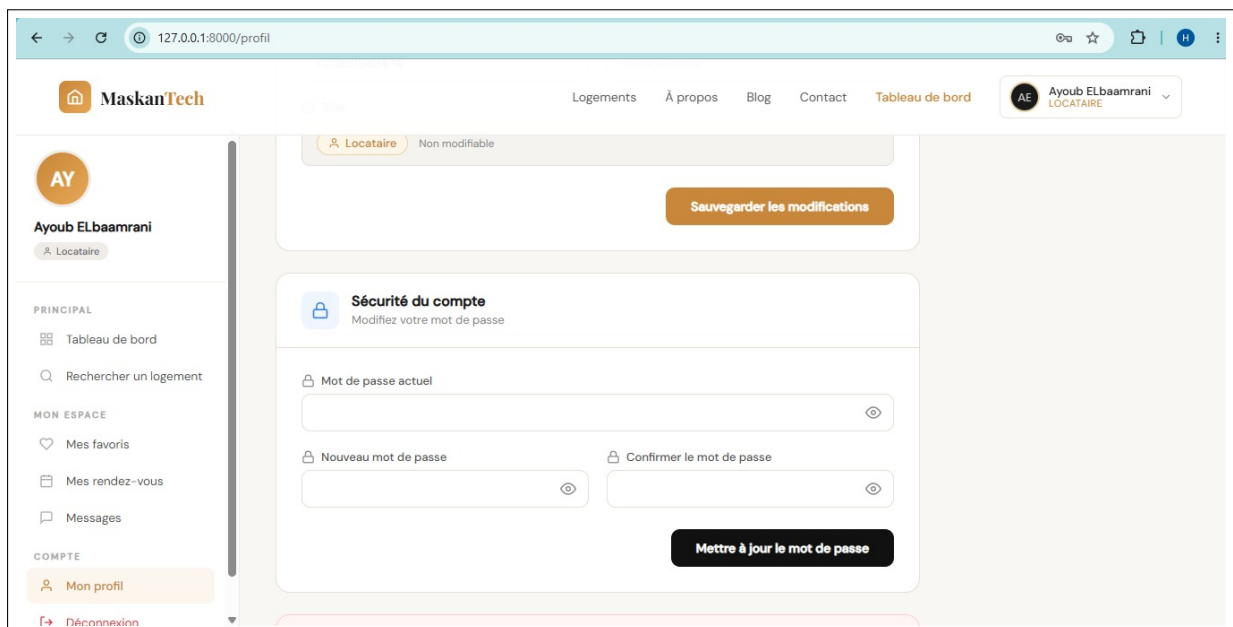
5.2.3 Réinitialisation du mot de passe

Le flux de réinitialisation du mot de passe est entièrement géré par Laravel Breeze. L'utilisateur saisit son adresse email, reçoit un lien de réinitialisation valable 60 minutes, puis définit un nouveau mot de passe via un formulaire sécurisé.



The screenshot displays the 'Mon profil' (My profile) page of the MaskanTech application. The page is titled 'Mon profil' and includes a subtitle 'Gérez vos informations personnelles et vos paramètres de sécurité'. The main content area is divided into sections: 'Informations personnelles' (Personal information) and 'Photo de profil' (Profile photo). The 'Informations personnelles' section contains fields for 'Nom complet' (Full name), 'Adresse email' (Email address), 'Téléphone' (Phone), and 'Adresse' (Address). The 'Photo de profil' section shows a profile picture placeholder and a 'Changer la photo' (Change photo) button. A sidebar on the left contains navigation links: 'Tableau de bord' (Dashboard), 'Rechercher un logement' (Search for a property), 'Mes favoris' (My favorites), 'Mes rendez-vous' (My appointments), 'Messages', and 'Mon profil' (My profile). The top navigation bar includes links for 'Logements' (Properties), 'À propos' (About), 'Blog', 'Contact', and 'Tableau de bord' (Dashboard). The user's name 'Ayoub ELbaamrani' and role 'LOCATAIRE' (Tenant) are displayed in the top right corner.

FIGURE 5.3 – Formulaire de réinitialisation du mot de passe



The screenshot displays the 'Sécurité du compte' (Account security) page of the MaskanTech application. The page is titled 'Sécurité du compte' and includes a subtitle 'Modifiez votre mot de passe' (Change your password). The main content area contains a form for updating the password, with fields for 'Mot de passe actuel' (Current password), 'Nouveau mot de passe' (New password), and 'Confirmer le mot de passe' (Confirm password). A 'Sauvegarder les modifications' (Save changes) button is visible. A sidebar on the left contains navigation links: 'Tableau de bord' (Dashboard), 'Rechercher un logement' (Search for a property), 'Mes favoris' (My favorites), 'Mes rendez-vous' (My appointments), 'Messages', and 'Mon profil' (My profile). The top navigation bar includes links for 'Logements' (Properties), 'À propos' (About), 'Blog', 'Contact', and 'Tableau de bord' (Dashboard). The user's name 'Ayoub ELbaamrani' and role 'LOCATAIRE' (Tenant) are displayed in the top right corner.

FIGURE 5.4 – Formulaire de réinitialisation du mot de passe

5.3 Interfaces publiques

5.3.1 Page principale — Liste des biens

La page de liste des biens (`/biens`) constitue l'interface principale de . Elle est accessible sans authentification et présente toutes les annonces disponibles en grille de trois colonnes, avec une pagination de douze biens par page.

La mise en page adopte un layout à deux colonnes : une sidebar de filtres sur la gauche et la grille de résultats sur la droite. Cette disposition permet à l'utilisateur d'affiner sa recherche sans perdre de vue les résultats.

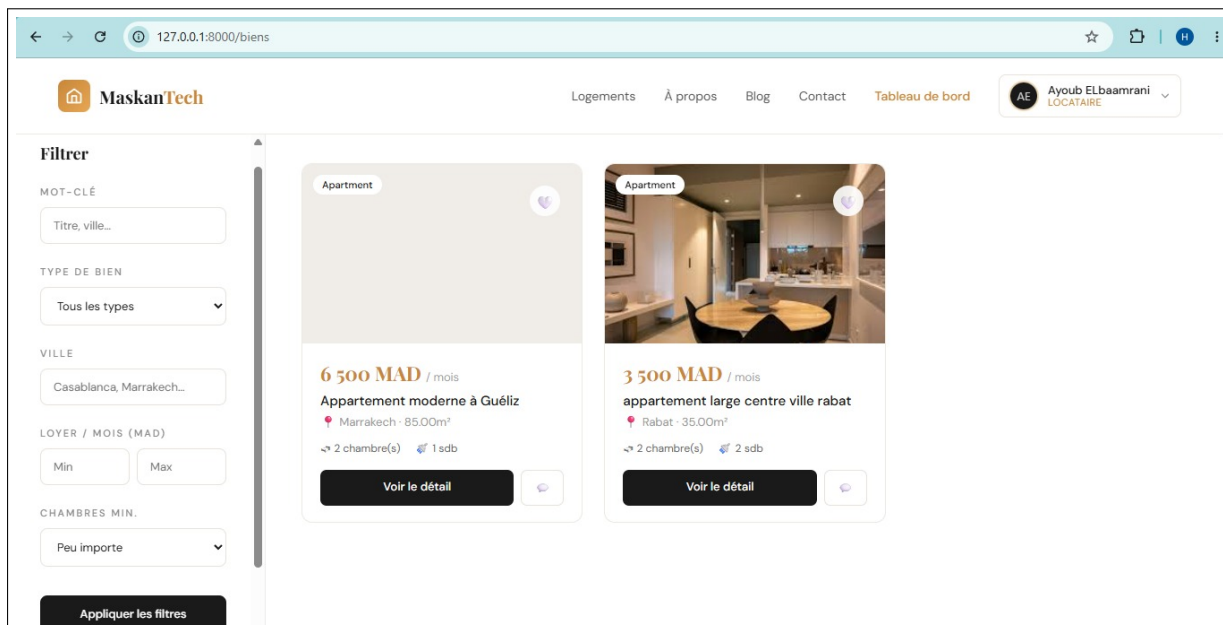


FIGURE 5.5 – Page principale de liste des biens immobiliers — `/biens`

Chaque carte de bien affiche :

- L'image principale du bien (ou un placeholder si aucune image n'a été uploadée).
- Le titre de l'annonce, la ville et le type de bien.
- Le prix en dirhams (MAD), la surface en m^2 et le nombre de chambres.
- Un badge de statut coloré : vert pour « Disponible », rouge pour « Vendu », orange pour « Loué » et bleu pour « En construction ».
- Un bouton cœur pour ajouter le bien aux favoris (accessible uniquement aux utilisateurs connectés ; redirige vers la connexion sinon).

5.3.2 Sidebar de filtres

La sidebar gauche de la page de liste offre un système de filtres complet permettant d'affiner les résultats selon plusieurs critères combinables.

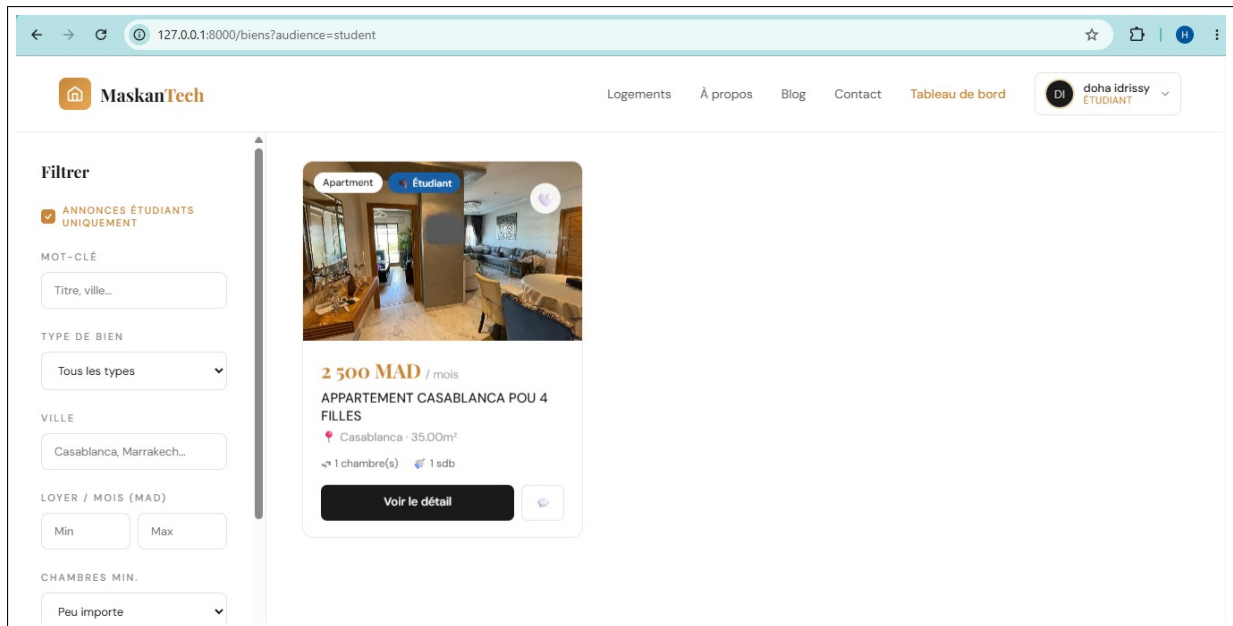


FIGURE 5.6 – Sidebar de filtres avancés de

Les filtres disponibles sont :

- **Recherche textuelle** : mot-clé recherché dans le titre, la description et l'adresse des biens.
- **Type de bien** : liste déroulante (maison, appartement, terrain, bureau).
- **Ville** : champ texte avec autocomplétion via l'API REST `/api/cities`.
- **Fourchette de prix** : deux champs numériques (prix minimum et prix maximum en MAD).
- **Nombre de chambres minimum** : liste déroulante (1, 2, 3, 4, 5+).
- **Statut** : case à cocher (disponible, loué, vendu, en construction).
- **Tri** : prix croissant, prix décroissant, date d'ajout, surface décroissante.

Tous les filtres sont transmis via la méthode GET, ce qui permet de partager l'URL de la recherche ou de la retrouver dans l'historique du navigateur. Le scope `filter()` du modèle `Property` prend en charge l'application dynamique des filtres selon les paramètres présents dans la requête.

5.3.3 Page de détail d'un bien

La page de détail (`/biens/{slug}`) présente l'ensemble des informations d'une annonce immobilière. Elle est structurée en plusieurs sections complémentaires.

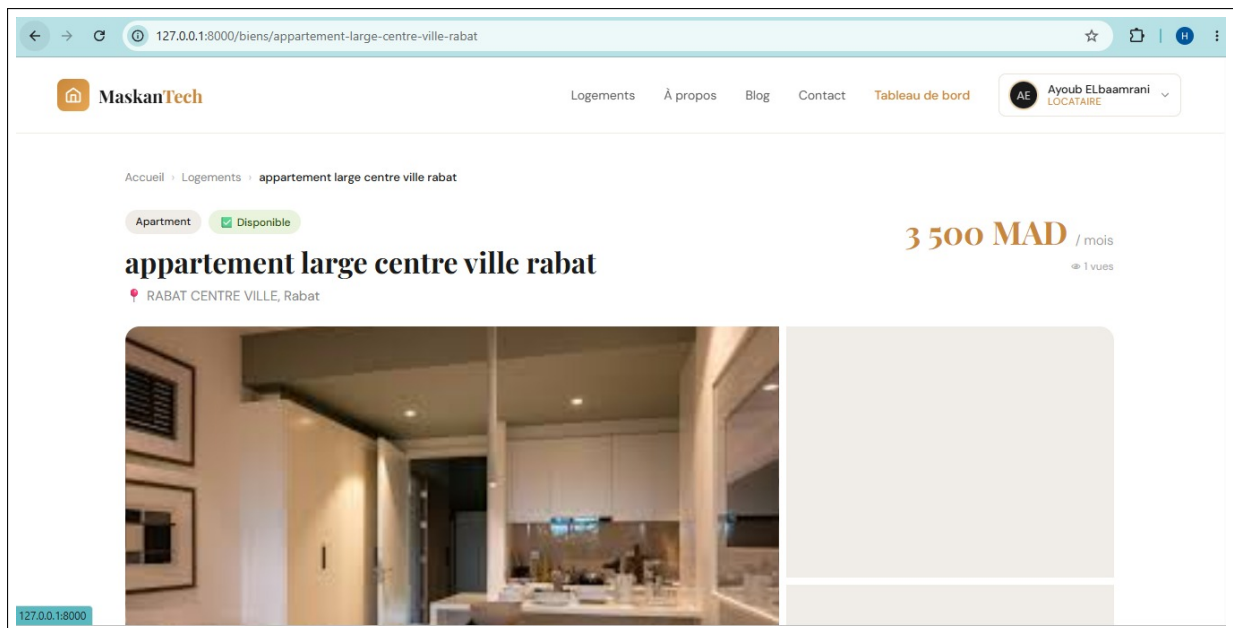


FIGURE 5.7 – Page de détail d’une annonce immobilière — `/biens/{slug}`

La page est organisée comme suit :

Galerie photos interactive Affichage de l’image principale en grand format avec navigation entre les images via Alpine.js. Un indicateur de position (1/5, 2/5...) informe l’utilisateur de sa position dans la galerie.

En-tête de l’annonce Titre, badge de statut, type de bien et date de publication. Le compteur de vues est affiché et incrémenté via une requête AJAX POST sur `/biens/{slug}/vue` au chargement de la page.

Caractéristiques principales Prix mis en évidence, surface, nombre de pièces, chambres et salles de bain présentés sous forme d’icônes.

Description Texte complet de l’annonce, affiché avec la mise en forme préservée.

Équipements et caractéristiques Liste des caractéristiques sélectionnées lors de la publication (piscine, garage, jardin, etc.) sous forme de badges.

Localisation Adresse complète, ville, code postal et année de construction.

Vidéo de présentation Si un lien YouTube ou Vimeo a été renseigné, la vidéo est affichée en embed responsive.

Fiche agent Avatar, nom et rôle de l’agent ayant publié l’annonce, avec un bouton de contact direct vers la messagerie.

Formulaire de demande de visite Accessible uniquement aux clients connectés. Permet de choisir une date (dans le futur), une heure et d’ajouter un message optionnel.

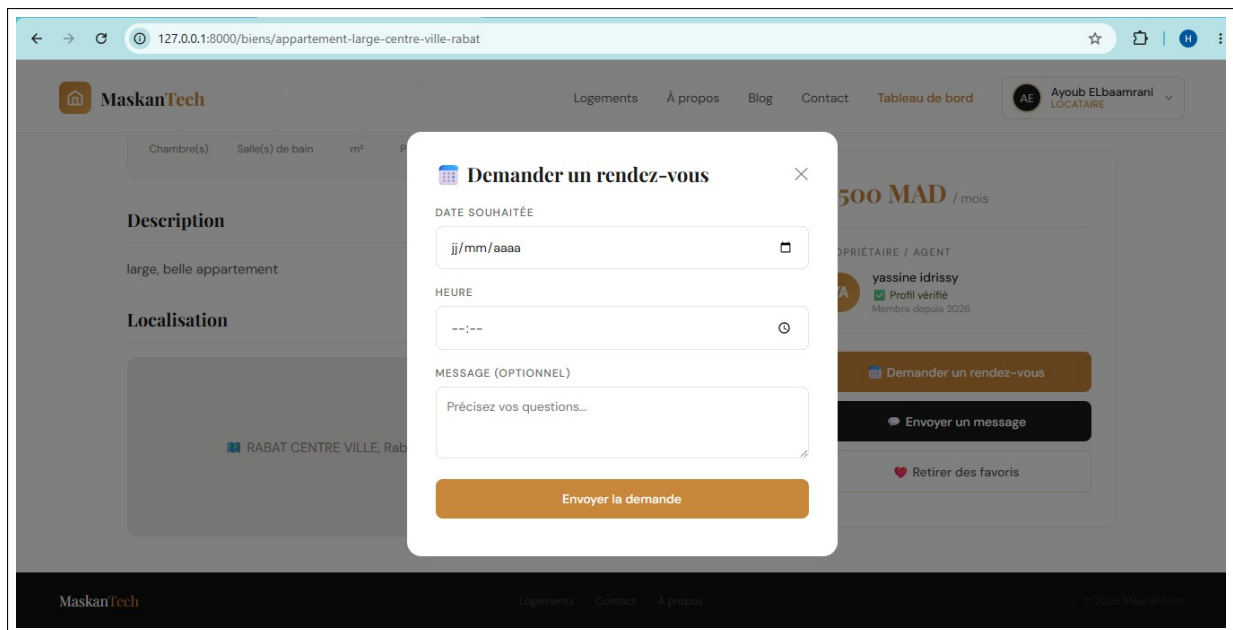


FIGURE 5.8 – Section formulaire de demande de visite — page de détail

5.3.4 Page de contact

La page de contact (`/contact`) est accessible sans authentification et permet à tout visiteur d'envoyer un message à l'équipe de . Le formulaire collecte le nom, l'adresse email, le numéro de téléphone (optionnel), le sujet et le message.

Les messages reçus sont enregistrés dans la table `contacts` et consultables par l'administrateur dans son interface de gestion.

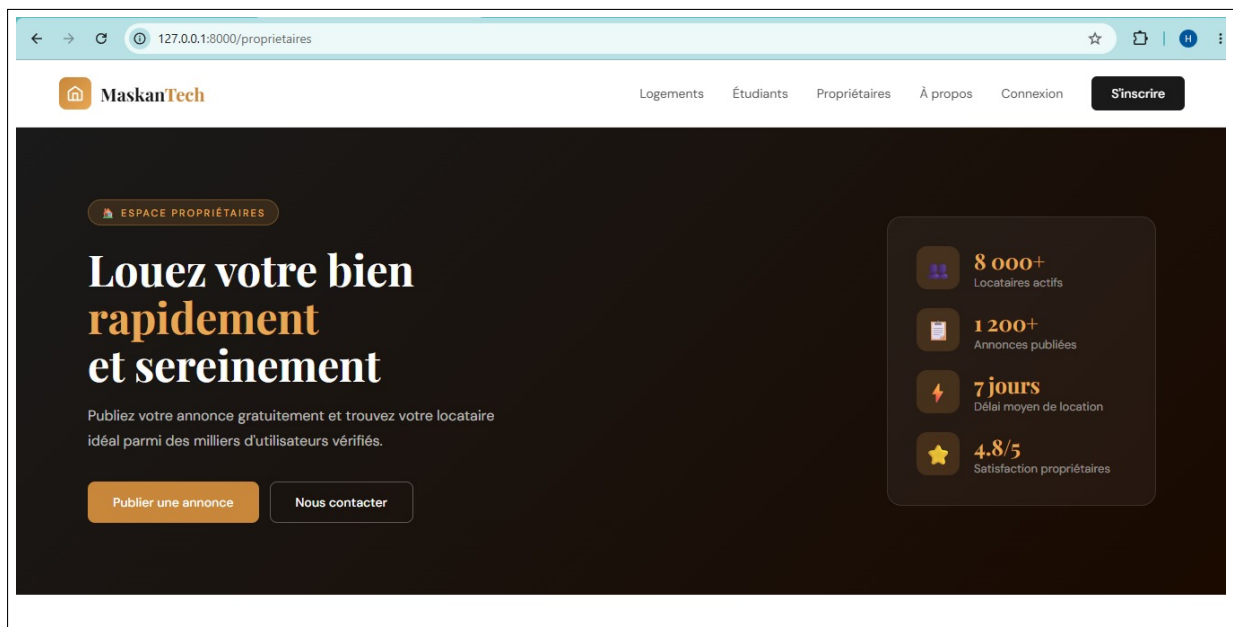


FIGURE 5.9 – Page de contact public — `/contact`

5.4 Interfaces locataire & Etudiant

5.4.1 Dashboard Etudiant

Le tableau de bord locataire (/dashboard) est le premier écran affiché après la connexion d'un utilisateur ayant le rôle `client`. Il offre une vue synthétique de l'activité de l'utilisateur sur la plateforme.

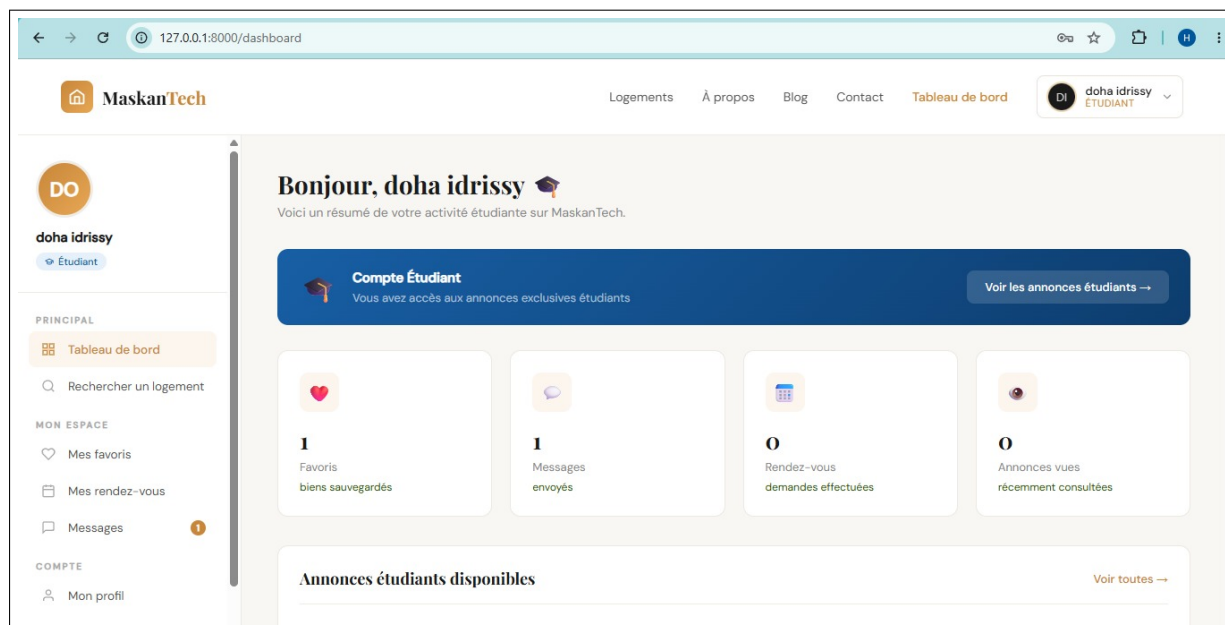


FIGURE 5.10 – Tableau de bord du locataire — /dashboard

Le dashboard locataire comprend quatre métriques résumées en haut de page : le nombre de biens mis en favoris, le nombre de demandes de visite effectuées, le nombre de messages envoyés et le nombre de biens récemment consultés.

En dessous, trois sections distinctes sont présentées :

- **Mes favoris** : grille des six derniers biens sauvegardés, avec un lien vers la liste complète des favoris.
- **Mes rendez-vous** : liste des dix dernières demandes de visite avec le statut de chacune (en attente, confirmé, refusé, terminé) et les coordonnées du bien et du propriétaire.
- **Biens récemment consultés** : liste des derniers biens visités, stockée en session côté serveur et mise à jour à chaque consultation de page de détail.

5.4.2 Page des favoris

La page des favoris (/favoris) affiche l'ensemble des biens sauvegardés par le locataire, présentés sous forme de grille paginée identique à la page de liste principale.

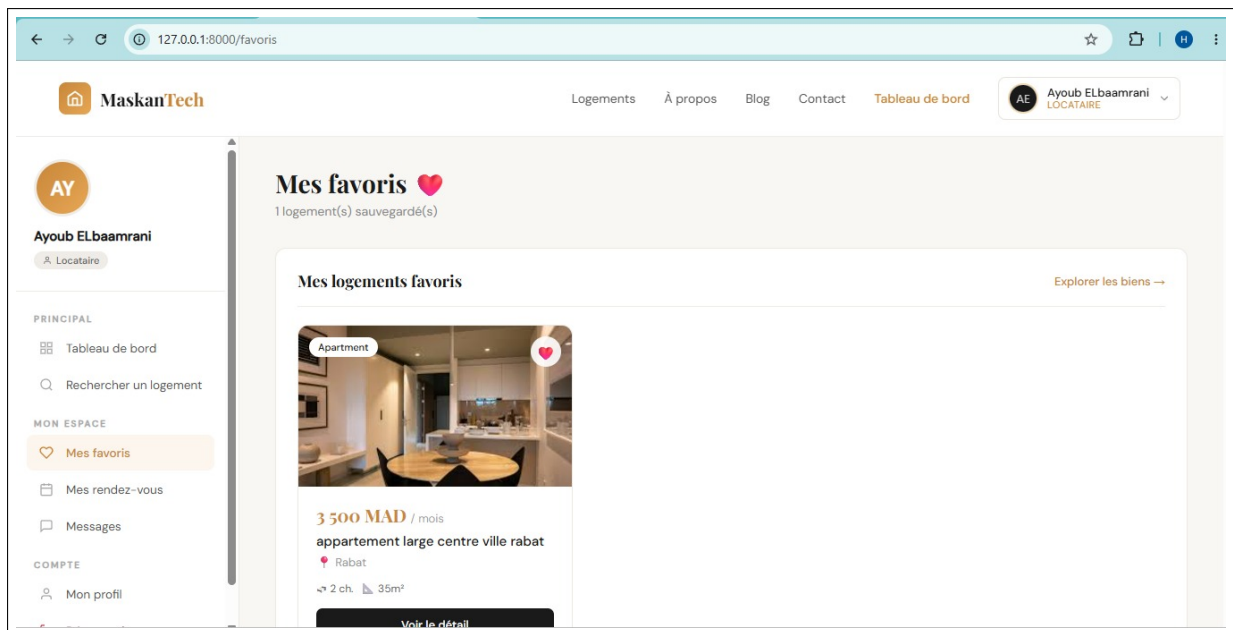


FIGURE 5.11 – Page des favoris du locataire — /favoris

L'ajout et la suppression d'un bien des favoris s'effectuent via un bouton cœur présent sur chaque carte de bien (page de liste) et sur la page de détail. L'action est envoyée en POST vers `/favoris/{property}/toggle` et fonctionne comme un toggle : si le bien est déjà en favoris, il est retiré ; sinon, il est ajouté.

5.4.3 Page des rendez-vous (locataire)

La page des rendez-vous (`/rendez-vous`) affiche pour le locataire la liste de toutes ses demandes de visite, passées et à venir, avec leur statut respectif.

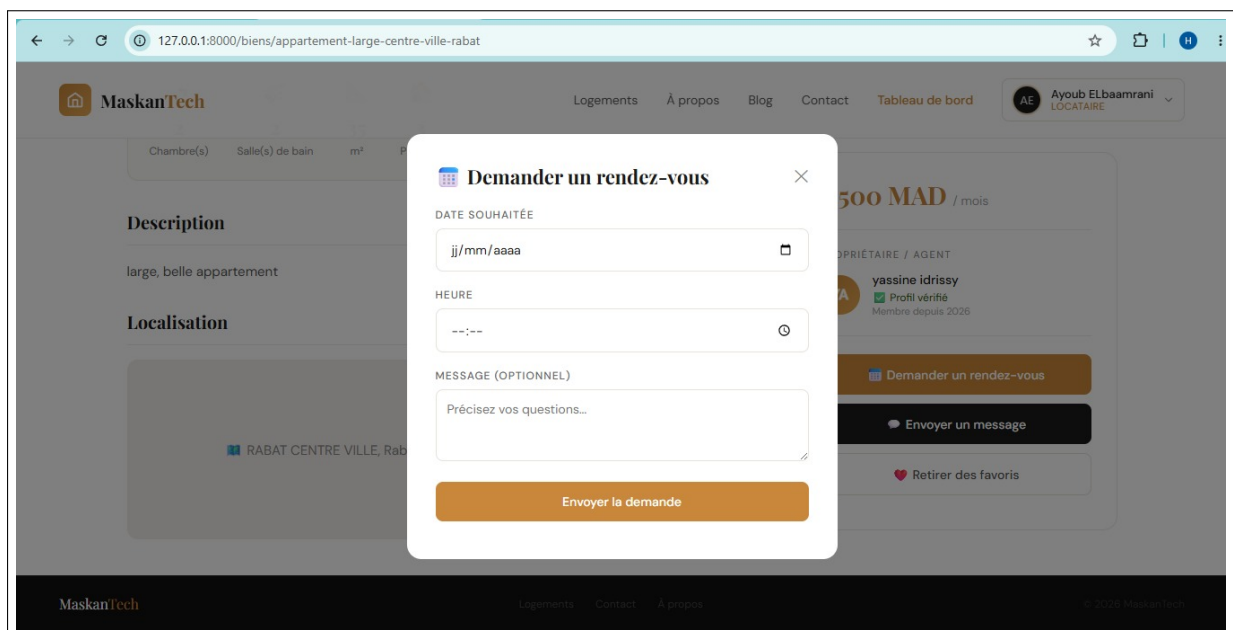


FIGURE 5.12 – rendez-vous du locataire — /rendez-vous

Les statuts sont représentés par des badges colorés :

- **En attente** (jaune) : la demande a été envoyée, le propriétaire ne l'a pas encore traitée.
- **Confirmé** (vert) : le propriétaire a accepté la demande.
- **Refusé** (rouge) : le propriétaire a refusé la demande.
- **Terminé** (gris) : la visite a eu lieu.

Le locataire peut annuler une demande en attente en cliquant sur le bouton de suppression, qui envoie une requête DELETE vers `/rendez-vous/{appointment}`.

5.4.4 Messagerie — Vue locataire

Liste des conversations

La page de messagerie (`/messages`) affiche la liste de toutes les conversations du locataire, groupées par interlocuteur. Chaque conversation affiche le dernier message échangé, la date et un indicateur visuel si des messages non lus sont présents.

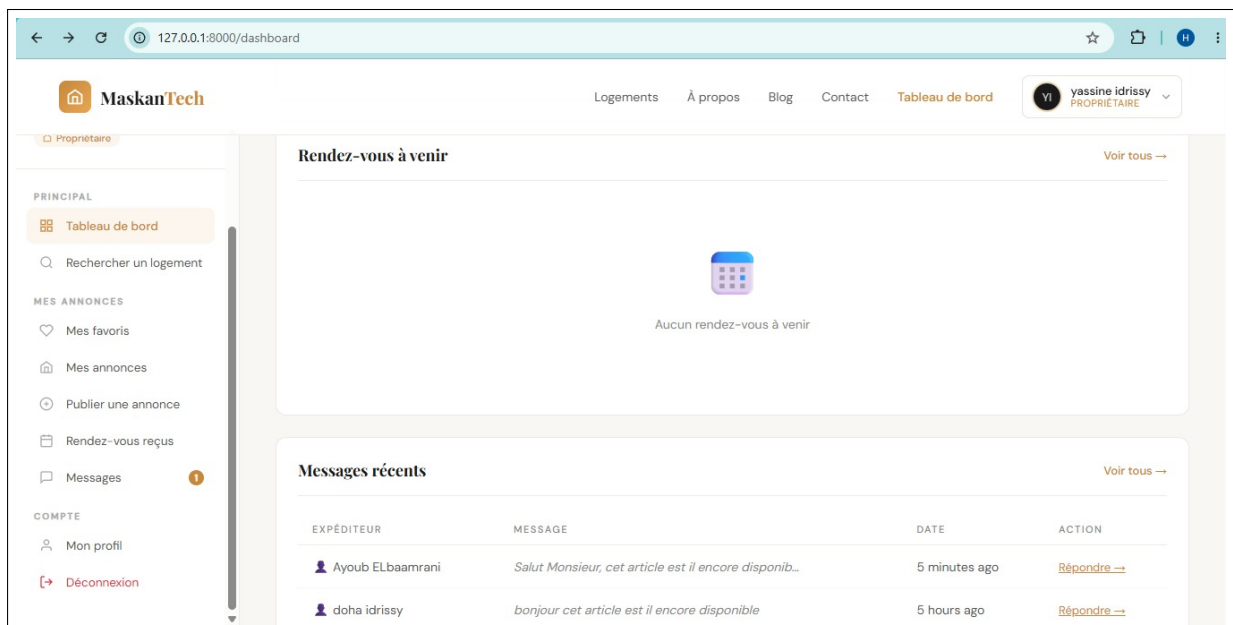


FIGURE 5.13 – Liste des conversations — `/messages`

Conversation avec un propriétaire

La page de conversation (`/messages/{user}`) affiche l'historique complet des échanges entre le locataire et un propriétaire, dans un format similaire à une messagerie instantanée. Les messages de l'utilisateur courant sont alignés à droite, ceux de l'interlocuteur à gauche.

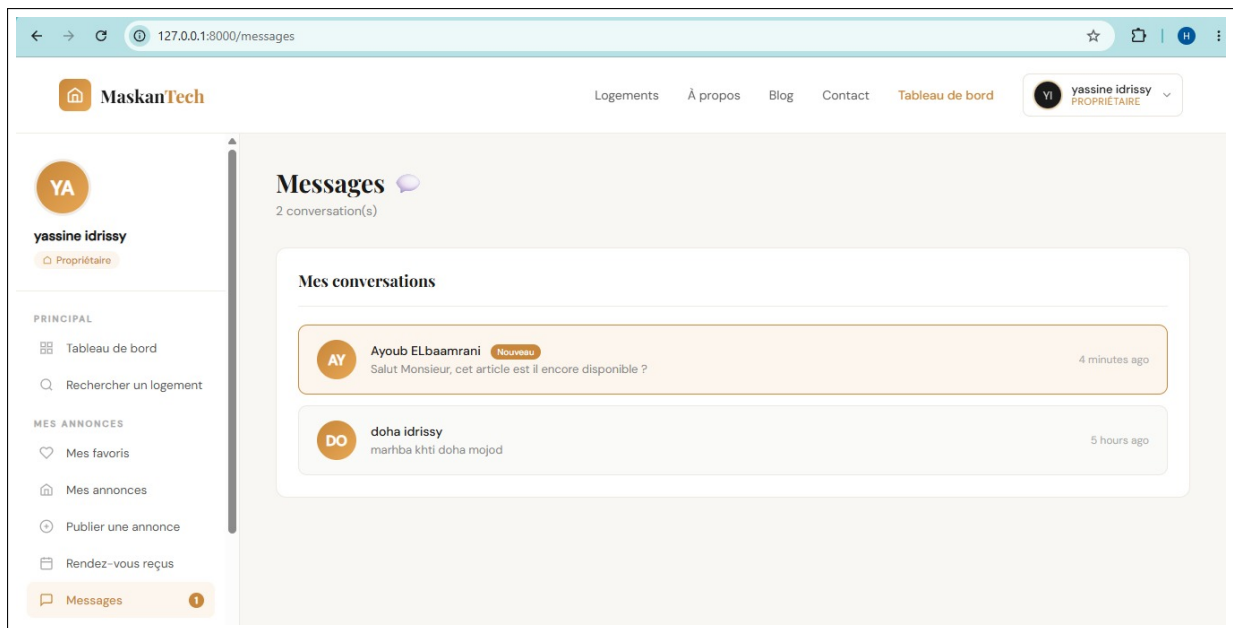


FIGURE 5.14 – Conversation avec un propriétaire — `/messages/{user}`

Lors de l'accès à une conversation, tous les messages reçus non lus sont automatiquement marqués comme lus via une requête de mise à jour en masse (`update(['is_read' => true])`), ce qui met à jour le compteur de la navbar en temps réel.

5.4.5 Page de profil

La page de profil (`/profil`) permet à l'utilisateur de modifier ses informations personnelles : nom, numéro de téléphone, adresse et photo de profil.

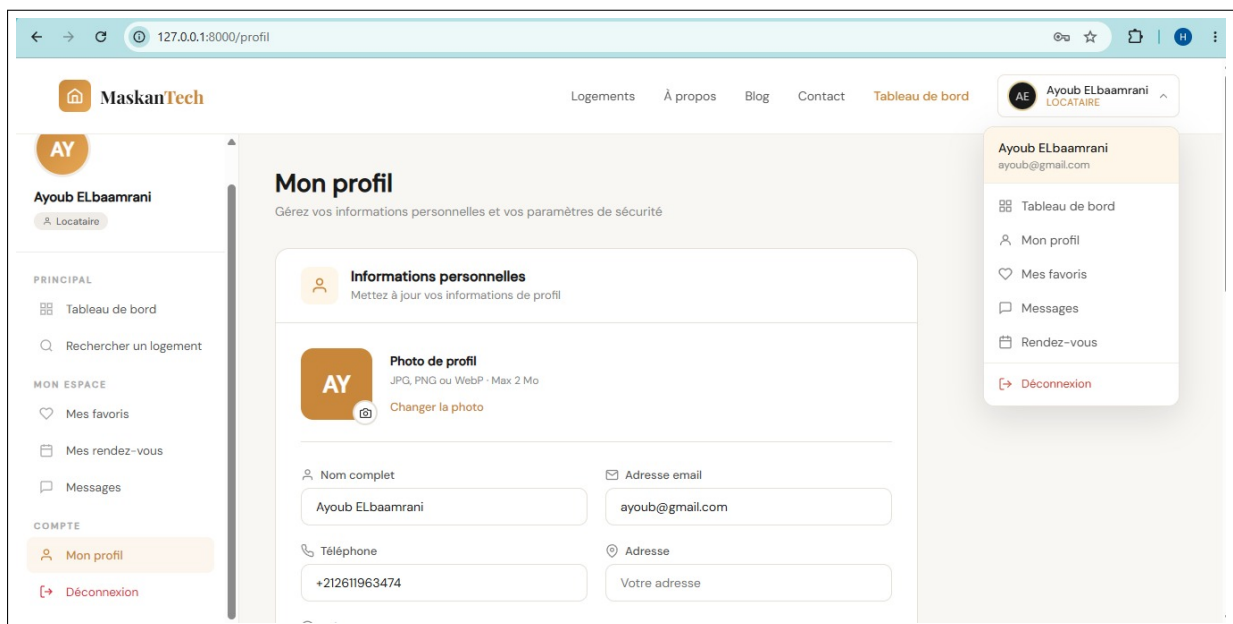


FIGURE 5.15 – Page d'édition du profil utilisateur — `/profil`

L'upload de l'avatar est géré par le `ProfileController` : l'image est stockée dans le dossier `storage/app/public/avatars/` et son chemin est enregistré dans la colonne

avatar de la table `users`. Si aucun avatar n'est défini, un avatar généré dynamiquement à partir des initiales de l'utilisateur est affiché via l'API `ui-avatars.com`.

5.5 Interfaces propriétaire

5.5.1 Dashboard propriétaire

Le tableau de bord propriétaire est affiché lorsqu'un utilisateur ayant le rôle `agent` se connecte et accède à `/dashboard`. Il centralise toutes les informations pertinentes pour la gestion professionnelle de ses annonces.

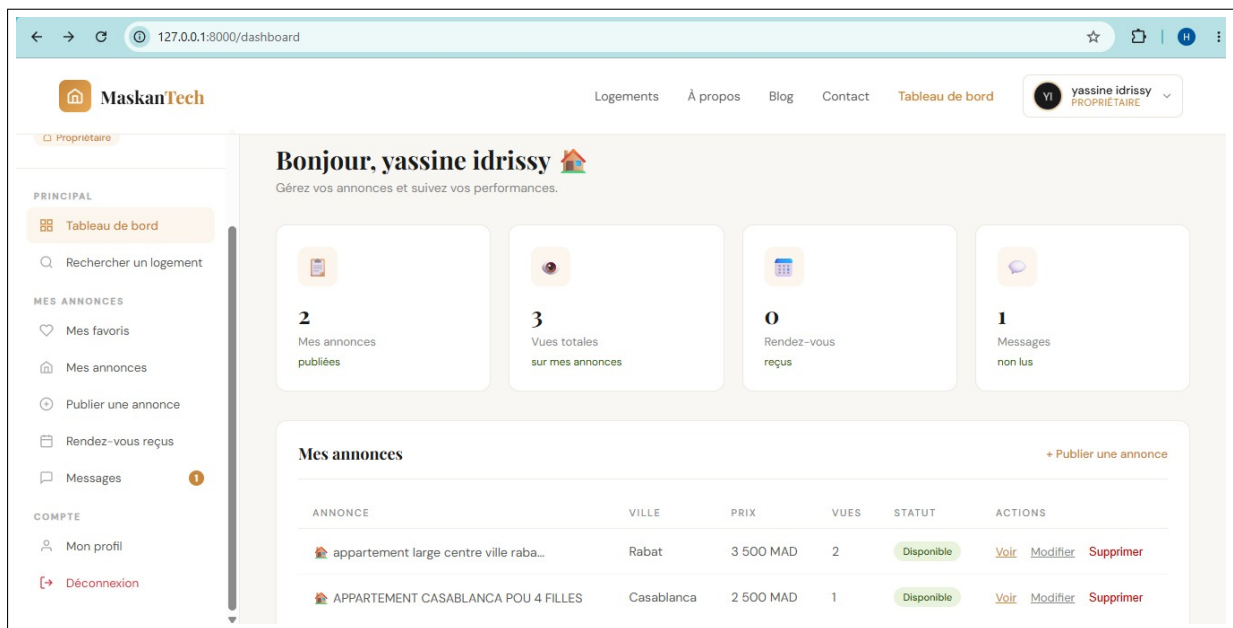


FIGURE 5.16 – Tableau de bord du propriétaire — `/dashboard`

Le dashboard propriétaire présente :

Métriques personnelles (4 cartes) Nombre de biens publiés, total des vues cumulées sur toutes les annonces, nombre de rendez-vous reçus ce mois, nombre de fois où ses biens ont été mis en favoris par des locataires.

Mes annonces Liste paginée des biens publiés par le propriétaire, avec image principale, titre, prix, statut, nombre de vues et actions rapides (modifier, supprimer).

Prochains rendez-vous Les cinq prochains rendez-vous confirmés ou en attente, avec la date, l'heure, le bien concerné et le nom du locataire demandeur.

Messages non lus Indicateur dans la navbar et dans le dashboard du nombre de messages des locataires non encore consultés.

5.5.2 Formulaire de publication d'une annonce

Le formulaire de publication (`/biens/creer`) est accessible uniquement aux propriétaires et administrateurs, grâce à la vérification du rôle via `abort_unless()` dans le `PropertyController`.

127.0.0.1:8000/biens/creer

MaskanTech Logements À propos Blog Contact Tableau de bord yassine idrissy PROPRIÉTAIRE

Détails du logement

Informations techniques sur votre bien.

SURFACE (M²) NOMBRE DE PIÈCES

Ex: 35 Ex: 3

CHAMBRES SALLES DE BAIN

Ex: 2 Ex: 1

Photos du logement

Ajoutez jusqu'à 10 photos. La première sera la photo principale.

FIGURE 5.17 – Formulaire de publication d'une annonce — /biens/creer

127.0.0.1:8000/biens/creer

MaskanTech Logements À propos Blog Contact Tableau de bord yassine idrissy PROPRIÉTAIRE

PUBLIER UNE ANNONCE

Décrivez votre bien

Remplissez les informations ci-dessous pour publier votre annonce gratuitement.

Informations générales

Décrivez votre bien en détail pour attirer les bons locataires.

TITRE DE L'ANNONCE

Ex: Studio meublé — Guéliz, Marrakech

TYPE DE BIEN LOYER MENSUEL (MAD)

Choisir un type Ex: 2500

VILLE CODE POSTAL

Choisir une ville Ex: 40000

ADRESSE COMPLÈTE

FIGURE 5.18 – Formulaire de publication d'une annonce — /biens/creer

Le formulaire est organisé en sections thématiques :

- **Informations générales** : titre, description, prix, surface.
- **Classification** : type de bien (maison/appartement/terrain/bureau), statut (disponible/vendu/loué/en construction), option « bien à la une ».
- **Détails** : nombre de pièces, chambres, salles de bain, année de construction.
- **Localisation** : adresse, ville, code postal.
- **Caractéristiques** : cases à cocher pour les équipements (piscine, garage, jardin, climatisation, chauffage, meublé, ascenseur, interphone, terrasse, cave).

- **Médias** : upload multiple d'images (jusqu'à 10, 5 Mo max chacune), lien vidéo YouTube ou Vimeo optionnel.

La validation est assurée par la classe `PropertyRequest` côté serveur. En cas d'erreur, les messages de validation s'affichent en rouge sous chaque champ concerné, et le formulaire conserve les valeurs saisies grâce au helper `old()` de Blade.

□

FIGURE 5.19 – Formulaire de modification d'une annonce — `/biens/{slug}/modifier`

5.5.3 Gestion des rendez-vous (propriétaire)

La page des rendez-vous vue par le propriétaire (`/rendez-vous`) affiche toutes les demandes de visite reçues pour ses biens, avec les coordonnées du locataire demandeur et les boutons d'action.

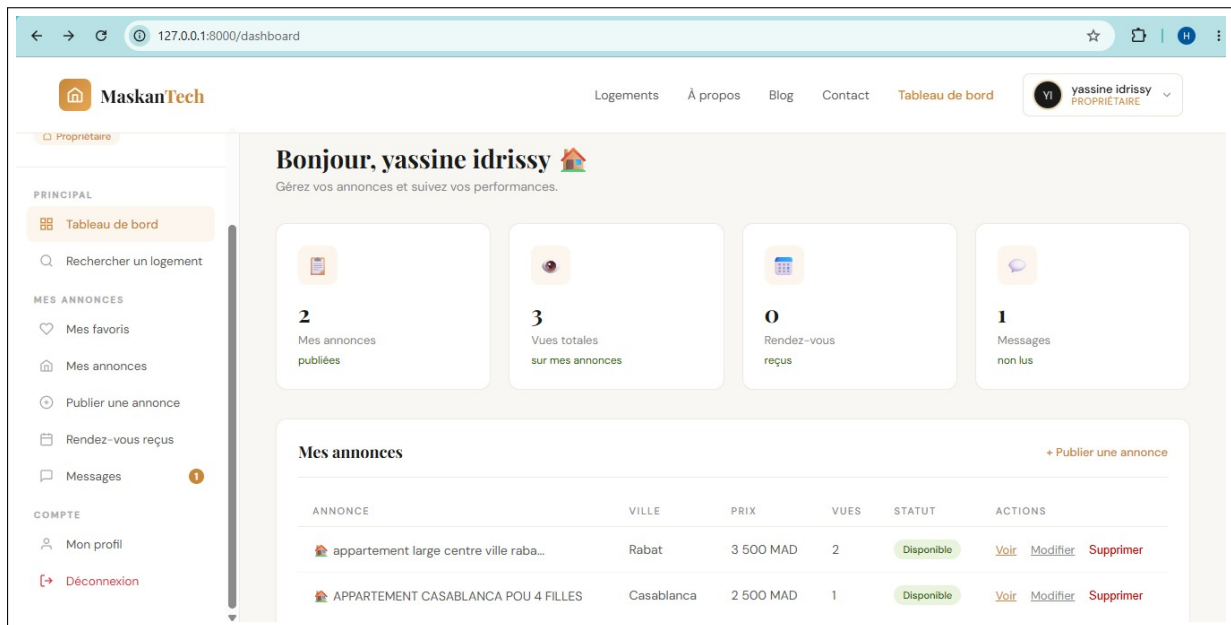


FIGURE 5.20 – Gestion des demandes de visite — vue propriétaire

Le propriétaire peut effectuer trois actions sur une demande :

- **Confirmer** : change le statut de `pending` à `confirmed` via une requête PATCH vers `/rendez-vous/{appointment}/statut`.
- **Refuser** : change le statut à `refused`.
- **Marquer comme terminé** : change le statut à `completed` une fois la visite effectuée.

5.6 Interfaces administrateur

5.6.1 Dashboard administrateur

Le tableau de bord administrateur (`/admin/`) est le centre de contrôle de la plateforme. Il offre une vue globale et en temps réel de l'activité de .

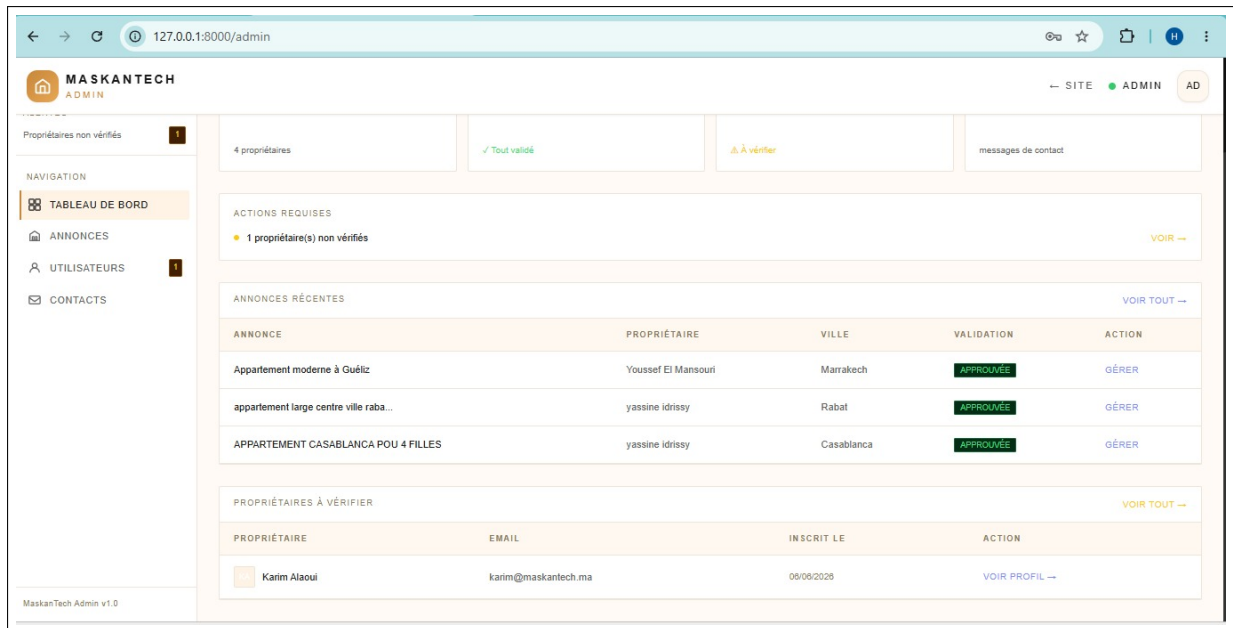


FIGURE 5.21 – Tableau de bord de l’administrateur — /admin/

Le dashboard administrateur est structuré en plusieurs zones :

Métriques globales (4 cartes) Total des biens publiés, nombre total d’utilisateurs inscrits, nombre de rendez-vous ce mois, nombre de ventes conclues (biens avec statut sold).

Graphique Chart.js Graphique en barres représentant le nombre de biens publiés par mois sur les douze derniers mois. Les données sont calculées dans le `DashboardController` et injectées dans la vue via la directive `@json($chartData)`.

Feed d’activité Flux des vingt derniers événements de la plateforme (nouvelles annonces, ventes, etc.), filtrable par type grâce à Alpine.js. Chaque élément affiche l’auteur, le bien concerné et la date relative.

Sidebar Liste des agents actifs avec leur statut, métriques de santé du système (RDV en attente, messages non lus, agents actifs) et section d’alertes.

5.6.2 Gestion des utilisateurs

La page de gestion des utilisateurs (/admin/utilisateurs) liste l’ensemble des comptes inscrits sur la plateforme, avec leurs informations et leur rôle actuel.

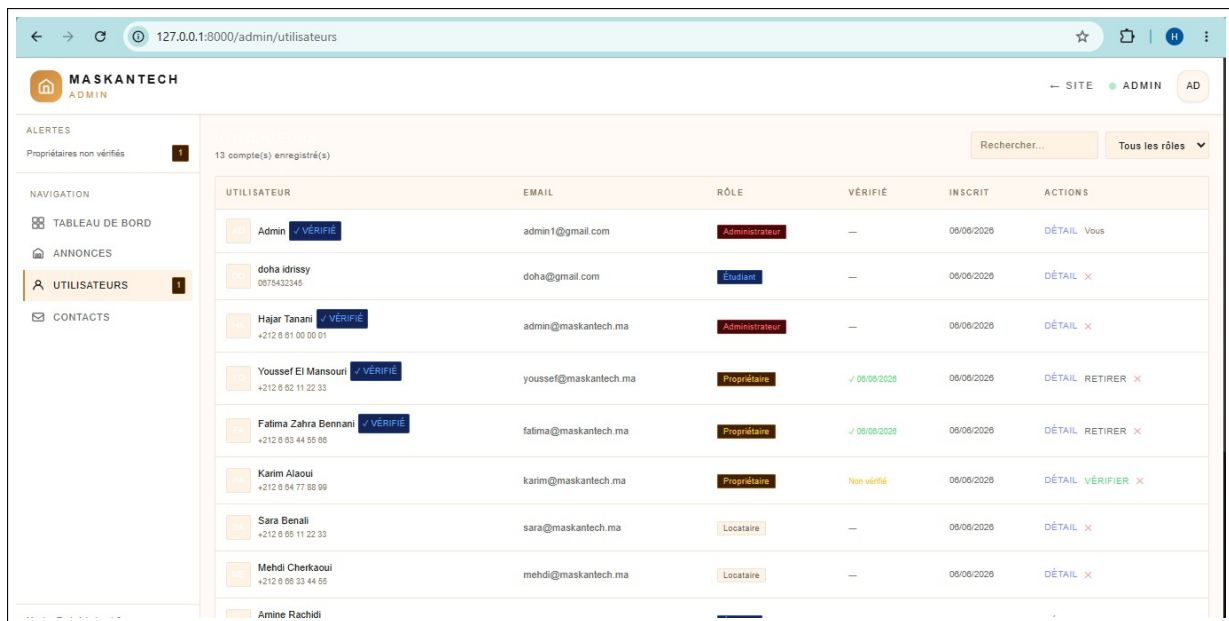


FIGURE 5.22 – Interface de gestion des utilisateurs — /admin/utilisateurs

L'administrateur peut effectuer les actions suivantes :

- **Modifier le rôle** d'un utilisateur via un menu déroulant (client ↔ agent ↔ admin), soumis via PATCH vers /admin/utilisateurs/{user}/role.
- **Supprimer un compte** via un bouton de suppression avec confirmation, soumis via DELETE vers /admin/utilisateurs/{user}.
- **Filtrer par rôle** via une liste déroulante permettant d'afficher uniquement les admins, les agents ou les clients.

Une sécurité est implémentée dans l'AdminController pour empêcher l'administrateur de se supprimer lui-même : `abort_if($user->id === auth()->id(), 403)`.

5.6.3 Gestion de tous les biens

La page /admin/biens donne à l'administrateur une vue complète sur l'ensemble des annonces publiées sur la plateforme, quel que soit l'agent auteur.

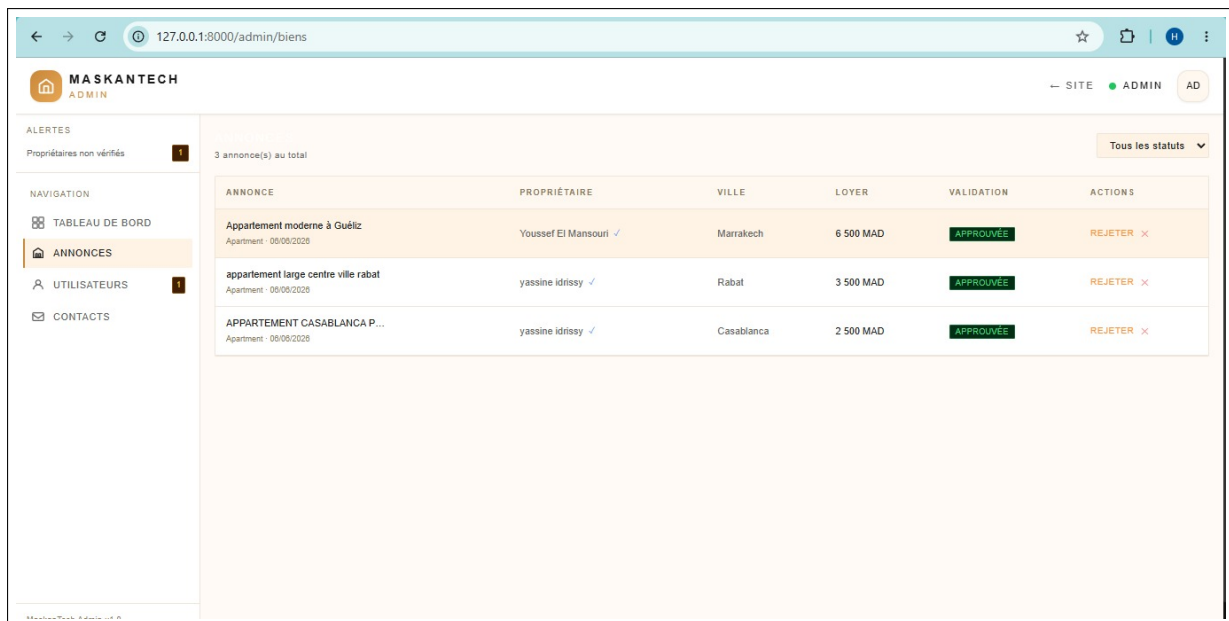


FIGURE 5.23 – Gestion de tous les biens immobiliers — /admin/biens

Contrairement aux agents qui ne voient que leurs propres biens, l'administrateur voit toutes les annonces. Il peut modifier ou supprimer n'importe quel bien, indépendamment de l'agent qui l'a publié. Cette capacité est assurée par la méthode `before()` de la `PropertyPolicy`, qui retourne `true` pour les administrateurs, court-circuitant toutes les autres vérifications d'autorisation.

5.6.4 Gestion des inscriptions propriétaires

La page `/admin/owners/verification` permet à l'administrateur de consulter la liste des demandes d'inscription des propriétaires en attente de validation. Chaque propriétaire doit fournir une pièce d'identité officielle (carte d'identité nationale, passeport ou titre de séjour) afin de prévenir les comptes anonymes ou frauduleux et d'assurer la traçabilité des annonces publiées sur la plateforme.

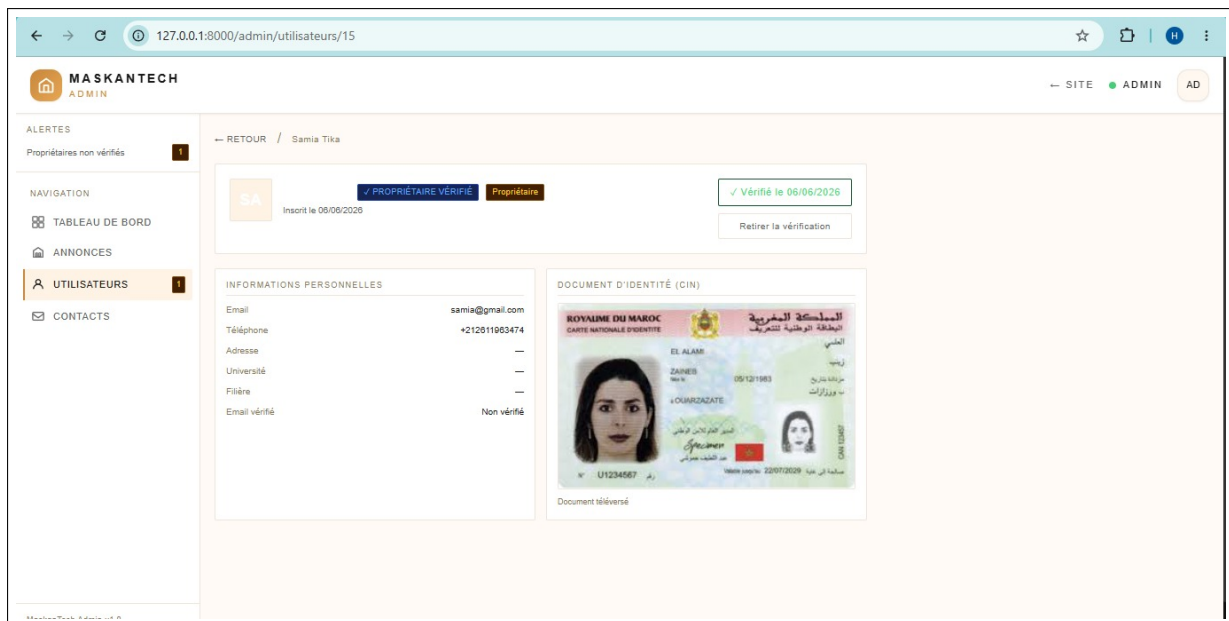


FIGURE 5.24 – Validation des propriétaires — /admin/owners/verification

Lors de l'accès à cette page, l'administrateur visualise pour chaque propriétaire son nom, son adresse e-mail, la date de sa demande, ainsi qu'un aperçu du document d'identité téléchargé. En cliquant sur le document, un modal ou un nouvel onglet s'ouvre pour un examen détaillé. Deux actions sont proposées :

- **Valider l'inscription** : L'administrateur confirme que la pièce d'identité est conforme et que le propriétaire est légitime. Le champ `owners.is_verified` passe à `true` et le propriétaire reçoit une notification par e-mail l'informant qu'il peut désormais publier des annonces.
- **Rejeter la demande** : L'administrateur peut optionnellement saisir un motif de refus (pièce illisible, expirée ou informations non concordantes). Le compte du propriétaire est alors désactivé ou supprimé selon la politique de la plateforme, et un e-mail de refus est envoyé.

Contrairement à un marquage automatique des messages de contact, cette approche impose une **validation humaine obligatoire** avant toute publication. Cela évite les inscriptions anonymes, réduit les risques d'arnaques ou de faux profils, et renforce la confiance des locataires dans les annonces diffusées. Toutes les actions de validation sont journalisées dans la table `admin_logs` pour une traçabilité complète.

5.6.5 Page d'accueil et présentation de l'équipe

La page d'accueil de est la première interface visible par les visiteurs non authentifiés. Elle a pour objectif de présenter la plateforme, ses fonctionnalités clés, ainsi que les acteurs à l'origine du projet.

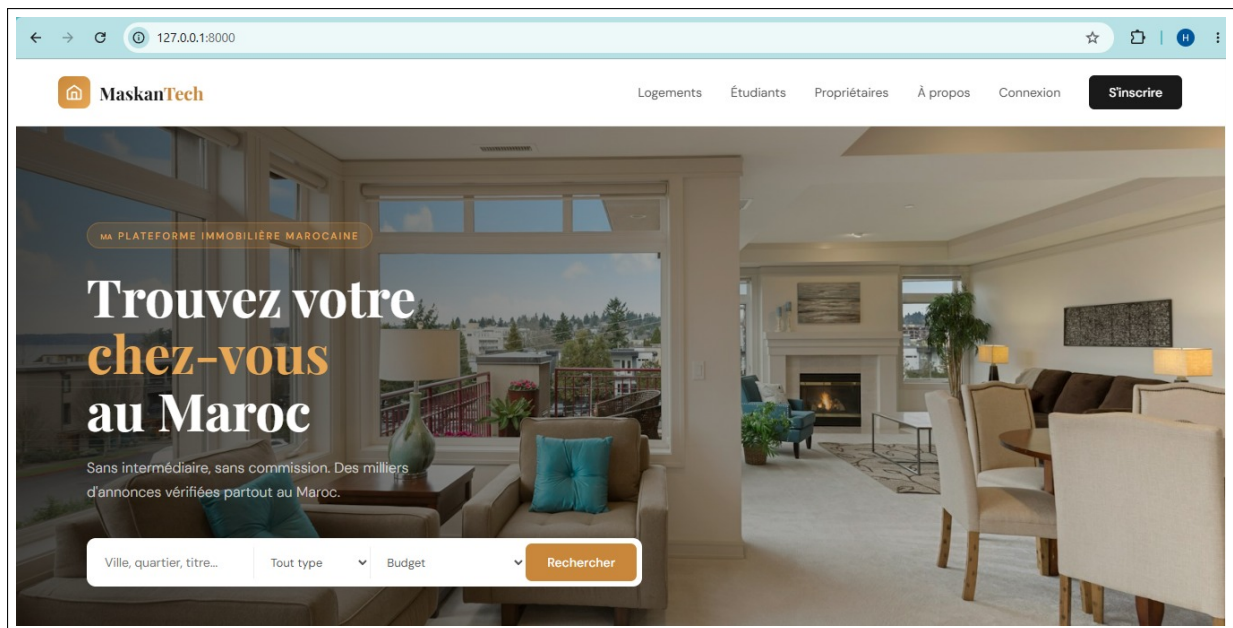


FIGURE 5.25 – Page d’accueil de

Présentation de la plateforme

La section principale de la page d’accueil présente un bref descriptif du service proposé par : mise en relation entre propriétaires et locataires, gestion simplifiée des rendez-vous pour les visites immobilières, et système de messagerie intégré. Trois cartes mettent en avant les fonctionnalités principales :

- **Publier une annonce** : dépôt d’annonces immobilières avec photos, description et critères de recherche.
- **Prendre un rendez-vous** : planification de visites directement via le calendrier intégré.
- **Messagerie sécurisée** : échange entre propriétaires et locataires sans partage d’informations personnelles.

Avis des réalisateurs

La seconde partie de la page d’accueil est consacrée à la présentation des deux réalisateurs du projet : **Elouzi** et **Tanani**. Cette section a pour vocation d’expliquer comment fonctionne la plateforme, les choix techniques effectués et la vision globale du projet.

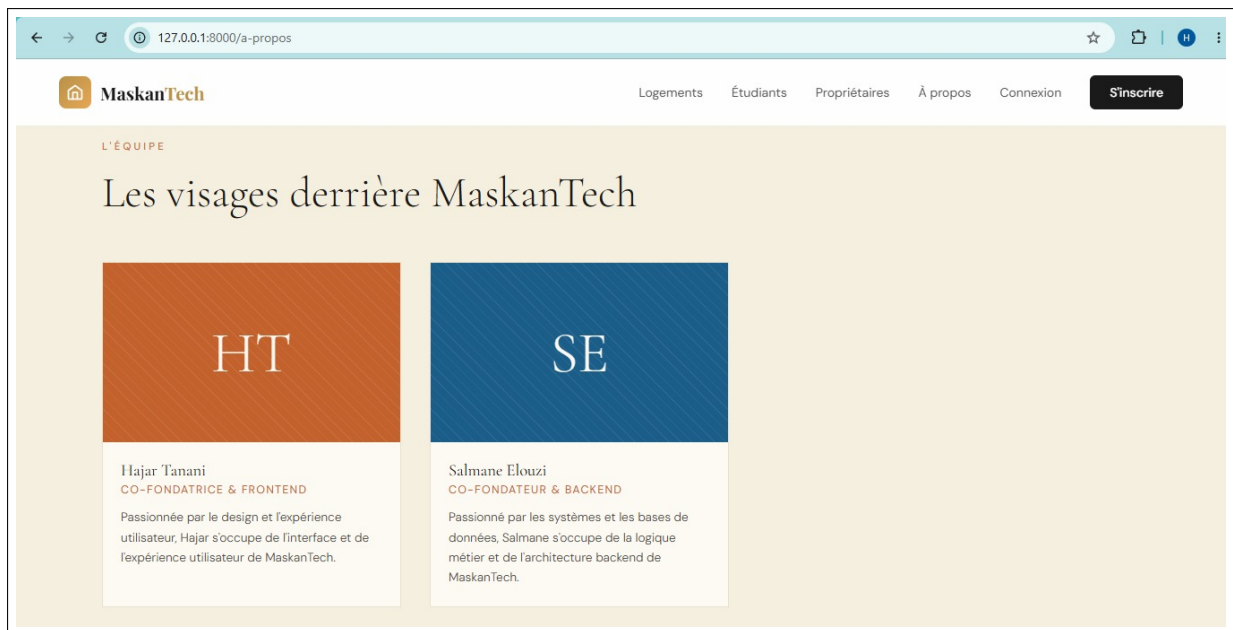


FIGURE 5.26 – Présentation de l'équipe — Elouzi & Tanani

Chaque réalisateur dispose d'un encart présentant :

- Son nom et son rôle dans le projet (chef de projet, développeur back-end, intégrateur front-end, etc.).
- Une brève biographie ou son parcours.
- Un avis personnel sur la réalisation du projet, les difficultés rencontrées et les solutions apportées.
- Les technologies maîtrisées et implémentées (Laravel, Alpine.js, Tailwind CSS, MySQL, etc.).

Un encart commun récapitule les grandes étapes du développement : analyse des besoins, conception de la base de données, développement des fonctionnalités principales (CRUD des biens, gestion des rendez-vous, messagerie, validation des propriétaires par pièce d'identité), phase de tests et déploiement.

À propos du projet : est une plateforme immobilière développée dans le cadre d'un projet académique. L'objectif était de concevoir une application web sécurisée, responsive et intuitive, permettant de gérer de bout en bout le cycle de vie d'une annonce immobilière, de la publication à la prise de rendez-vous, en passant par la validation des identités pour lutter contre l'anonymat frauduleux.

5.7 Conclusion

Ce chapitre a présenté l'ensemble des interfaces de à travers des captures d'écran commentées. La plateforme offre des espaces distincts et cohérents pour chaque type d'utilisateur, avec une navigation claire et un design system *dark terminal* appliqué de manière uniforme sur l'ensemble des pages.

L'ensemble des fonctionnalités définies dans le cahier des charges ont été implémentées avec succès : gestion multi-rôles, annonces immobilières avec galerie photos, recherche avancée par filtres, système de rendez-vous avec suivi du statut, messagerie interne, gestion

des favoris et tableaux de bord personnalisés par rôle. constitue ainsi une solution web complète et opérationnelle pour la gestion immobilière, prête à être déployée et enrichie de nouvelles fonctionnalités dans ses versions futures.

Chapitre 6

Conclusion générale

6.1 Synthèse des travaux

Ce projet de fin d'études nous a permis de mener à bien la conception et le développement de , une plateforme web de gestion immobilière complète et opérationnelle. En mobilisant un ensemble cohérent de technologies modernes — Laravel 13, PHP 8.5, MySQL, TailwindCSS et Alpine.js — nous avons réussi à construire une application robuste, sécurisée et ergonomique, répondant aux besoins réels des acteurs du marché immobilier marocain.

L'architecture MVC de Laravel, combinée à l'ORM Eloquent, a permis de structurer le code de manière claire et maintenable, avec une séparation nette entre la logique métier, la couche de données et la présentation. Le système de gestion multi-rôles, implémenté à travers un middleware personnalisé (`RoleMiddleware`) et des politiques d'autorisation (`PropertyPolicy`), assure une sécurité fine des accès en fonction du profil de chaque utilisateur.

Sur le plan fonctionnel, couvre l'ensemble du cycle de vie d'une transaction immobilière : publication d'annonces illustrées par des galeries photos, recherche multicritères avec filtres et tri, prise de rendez-vous de visite avec suivi du statut, messagerie interne entre utilisateurs, et tableaux de bord personnalisés par rôle offrant une vision synthétique de l'activité.

6.2 Apports personnels

Ce projet a représenté une expérience formatrice majeure à plusieurs niveaux. Sur le plan technique, il nous a permis de maîtriser le développement web full-stack avec Laravel, d'approfondir notre compréhension de la modélisation de bases de données relationnelles complexes, et d'acquérir une expérience concrète en gestion de version avec Git et GitHub.

Sur le plan méthodologique, la démarche itérative adoptée nous a appris à structurer un projet de grande envergure en sprints successifs, à anticiper les difficultés techniques et à adapter notre plan de travail en conséquence. La confrontation aux erreurs réelles — problèmes de compatibilité entre Laravel et PHP 8.5, gestion des migrations, débogage des routes — nous a préparés aux exigences du monde professionnel.

6.3 Limites identifiées

Dans sa version actuelle, présente plusieurs limites qu'il convient de reconnaître :

- La messagerie interne est asynchrone et ne propose pas encore de communication en temps réel via WebSockets.
- La plateforme ne dispose pas d'application mobile native.
- L'intégration cartographique permettant de visualiser la localisation des biens sur une carte interactive n'a pas été implémentée.
- Le système de notifications par email n'est pas encore connecté à une file d'attente (queue) pour l'envoi asynchrone.

6.4 Perspectives d'amélioration

Plusieurs évolutions significatives sont envisageables pour enrichir la plateforme dans ses versions futures :

- **Messagerie temps réel** : intégration de Laravel Reverb (WebSockets) pour une expérience de chat instantané entre clients et agents.
- **Application mobile** : développement d'une application Android et iOS via React Native ou Flutter, en s'appuyant sur l'API REST de .
- **Cartographie interactive** : intégration de Google Maps API ou Leaflet.js pour afficher la localisation précise des biens.
- **Intelligence artificielle** : recommandation de biens basée sur l'historique de navigation et les préférences de l'utilisateur, estimation automatique du prix au mètre carré.
- **Export et rapports** : génération de rapports PDF et exports Excel pour les agents et les administrateurs via les packages `barryvdh/laravel-dompdf` et `maatwebsite/excel`.
- **Système de notation** : permettre aux clients d'évaluer les agents immobiliers après une visite, renforçant ainsi la confiance et la transparence sur la plateforme.

En définitive, ce projet représente une base solide et extensible pour une plateforme immobilière numérique répondant aux standards actuels du développement web. Les compétences acquises au cours de sa réalisation nous préparent efficacement aux exigences du monde professionnel en matière de développement logiciel et de conduite de projets informatiques.

Références bibliographiques

- [1] HAUT-COMMISSARIAT AU PLAN. « Note de conjoncture immobilière ». Consulté en 2026. 2024. URL : <https://www.hcp.ma> (page 10).
- [2] LARAVEL TEAM. « Laravel Documentation — Version 13.x ». Consulté en 2026. 2026. URL : <https://laravel.com/docs/13.x>.
- [3] PHP GROUP. « PHP Manual ». Consulté en 2026. 2026. URL : <https://www.php.net/manual/fr/>.
- [4] ORACLE CORPORATION. « MySQL 8.0 Reference Manual ». Consulté en 2026. 2024. URL : <https://dev.mysql.com/doc/refman/8.0/en/>.
- [5] TAILWIND LABS. « TailwindCSS Documentation ». Consulté en 2026. 2026. URL : <https://tailwindcss.com/docs>.
- [6] ALPINE.JS TEAM. « Alpine.js Documentation ». Consulté en 2026. 2026. URL : <https://alpinejs.dev/>.
- [7] CHART.JS CONTRIBUTORS. « Chart.js Documentation ». Consulté en 2026. 2025. URL : <https://www.chartjs.org/docs/>.
- [8] VITE TEAM. « Vite Documentation ». Consulté en 2026. 2026. URL : <https://vitejs.dev/>.
- [9] Matt STAUFFER. « Laravel : Up and Running ». 3^e éd. O'Reilly Media, 2023.
- [10] Baron SCHWARTZ, Peter ZAITSEV et Vadim TKACHENKO. « High Performance MySQL ». 4^e éd. O'Reilly Media, 2022.
- [11] Martin FOWLER. « Patterns of Enterprise Application Architecture ». Addison-Wesley, 2003.
- [12] Adam FREEMAN. « Pro PHP 8 MVC ». Apress, 2021.
- [13] Erich GAMMA et al. « Design Patterns : Elements of Reusable Object-Oriented Software ». Addison-Wesley, 1994.
- [14] MUBAWAB. « Mubawab Maroc — Immobilier au Maroc ». Consulté en 2026. URL : <https://www.mubawab.ma>.
- [15] AVITO. « Avito Maroc — Petites annonces ». Consulté en 2026. URL : <https://www.avito.ma>.
- [16] SAROUTY. « Sarouty — Location et vente immobilière ». Consulté en 2026. URL : <https://www.sarouty.ma>.
- [17] LARACASTS. « Laracasts — Laravel Video Tutorials ». Consulté en 2026. URL : <https://laracasts.com>.

- [18] STACK OVERFLOW COMMUNITY. « Stack Overflow — Developer Q&A ». Consulté en 2026. URL : <https://stackoverflow.com>.
- [19] ELOUZI ET TANANI. « Maskan : Plateforme immobilière Laravel ». URL : <https://github.com/Elouzi-art/MaskanTech.git> (visité le 07/06/2026).